

A VISUAL TUTORIAL · NO EQUATIONS

# Transformers From Scratch

The working mechanism of modern large language models, explained one idea at a time.

**how a language model sees this sentence: ten tokens**

Trans form ers are the engine of modern LL Ms

# How to read this tutorial

## ONE IDEA

### One idea per slide

Each slide adds a single step. If a slide feels easy, that is the design: the hard parts are spread over many small steps.

## NO EQUATIONS

### Words, pictures, small numbers

Every mechanism is shown with boxes, charts and toy numbers. Nothing here needs algebra to follow.

## FROM SCRATCH

### No prior knowledge assumed

We start from a guessing game and finish with a full, working picture of a modern language model.

COLOUR CODE

the token doing the looking

tokens being looked at

results and outputs

position and special tokens

Notes in this handwriting point at the one thing to notice on a slide.

# Roadmap: twelve parts

01

## The prediction game

What a language model is asked to do

02

## Before transformers

Older ideas and where they got stuck

03

## Tokens

Chopping text into pieces a machine can count

04

## Embeddings

Turning each piece into a list of numbers

05

## Position

Telling the model what order things came in

06

## Attention

How words gather information from each other

07

## Inside a block

The thinking step, shortcuts and stabilisers

08

## Vectors to words

Scores, probabilities and choosing a token

09

## Training

How billions of dials get their settings

10

## The model family

Encoders, decoders and their sizes

11

## LLMs in use

Context windows, prompts, tools and limits

12

## Putting it together

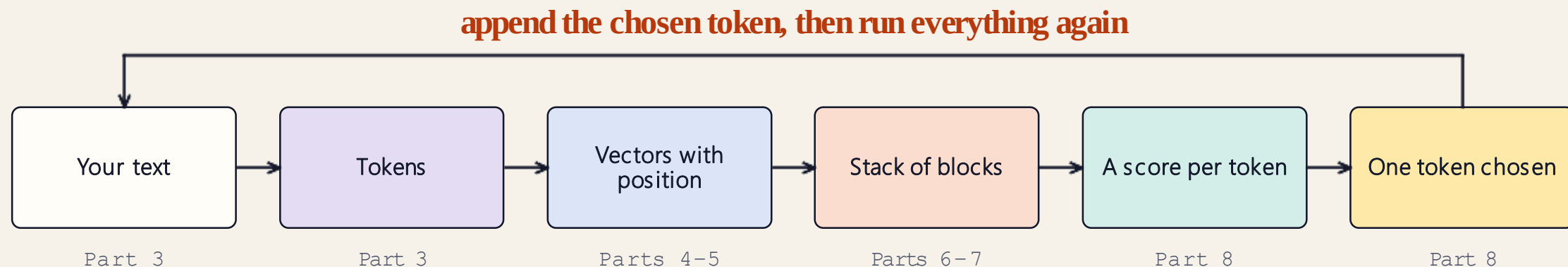
One prompt followed end to end

THE WHOLE IDEA IN ONE SENTENCE

A large language model is a machine that, again and again,  
predicts the next small piece of text.

Everything else in this deck explains how it makes that prediction so well.

# The whole machine on one slide



Text is cut into tokens, tokens become lists of numbers, a stack of identical blocks refines those numbers, and the final numbers become a score for every possible next token.

Each box is one or two parts of this deck. Keep this picture in mind; we will return to it at the end.

## OPENING

# Two sentences we will follow

SENTENCE A · USED FOR PREDICTION

The cat sat on the ?

Which token comes next? We will watch the model work this out, step by step.

SENTENCE B · USED FOR ATTENTION

The animal didn't cross the street because it was too tired

What does "it" refer to? Answering that is exactly the job attention was built for.

Tokens are drawn as boxes like these throughout the deck.

## PART 01

# The prediction game

A language model has one job: look at some text and guess what comes next. This part shows why that simple job is so demanding.

## IN THIS PART

The guessing game

Probability charts

Guessing needs knowledge

Generation as a loop

Counting versus learning

Neurons and layers

# Language modelling as a guessing game

A MESSAGE BEING TYPED

I' ll be there in five \_\_\_\_

SUGGESTIONS

minutes

mins

hours

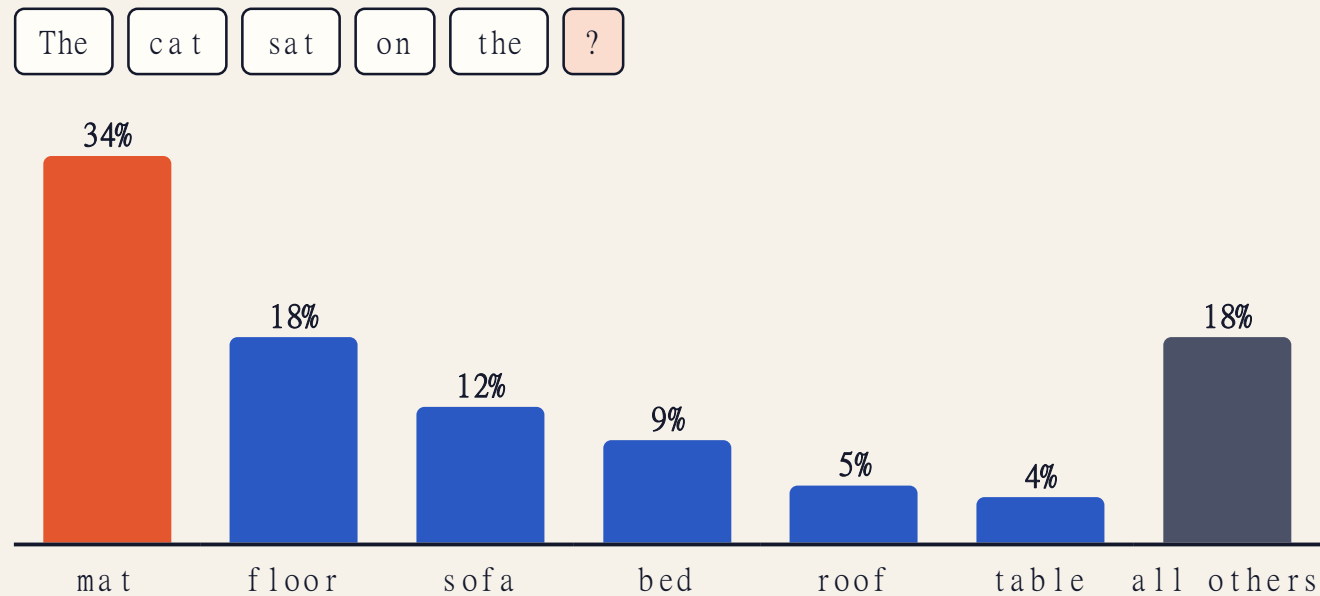
Given the words so far, what is the most likely next word?

Your phone keyboard plays this game every time you type. A large language model plays the same game, with far more skill and far more context.

**Hold on to this picture. Nothing a language model does goes beyond it.**



# A guess is a chart, not a single word



The model gives every token in its vocabulary a share of 100%.

Likely continuations get big shares. Unlikely ones get tiny shares, but almost never zero.

Numbers here are illustrative. A real chart has tens of thousands of bars.

# Why good guesses need knowledge

## GRAMMAR

The keys to the  
cabinet \_\_\_\_

needs are

## FACTS

The capital of France  
is \_\_\_\_

needs Paris

## COMMON SENSE

She dropped the glass  
and it \_\_\_\_

needs shattered

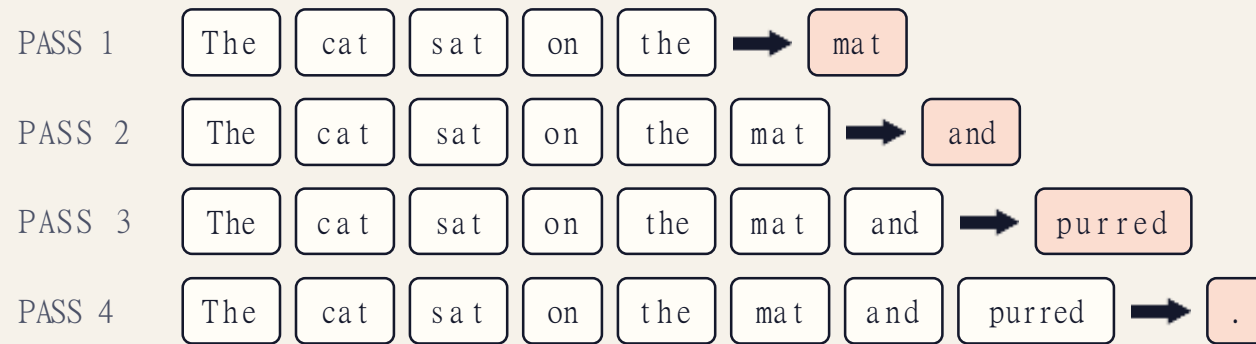
## REASONING

Tom is taller than  
Sam. Sam is taller than  
Lee. The shortest is  
\_\_\_\_

needs Lee

No one teaches the model grammar, facts or logic directly. It is only rewarded for guessing well, and guessing well across all kinds of text forces it to pick these up.

# Writing as repeated guessing

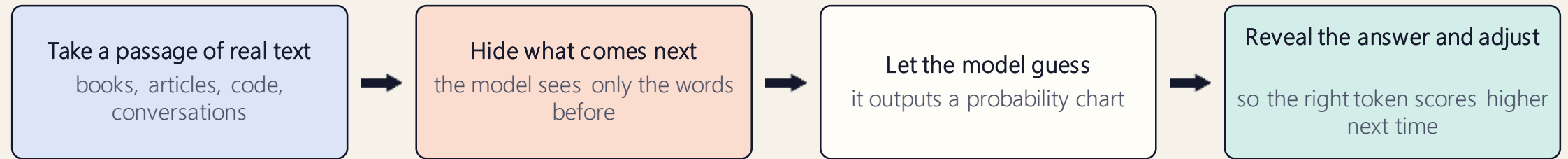


Each new token is glued onto the end, and the whole text goes back in as the next input.

This feed-it-back loop is called **autoregressive generation**. A long essay is hundreds of these passes in a row.

**The model never plans a whole sentence in one go. It only ever picks the next token.**

# Where the skill comes from

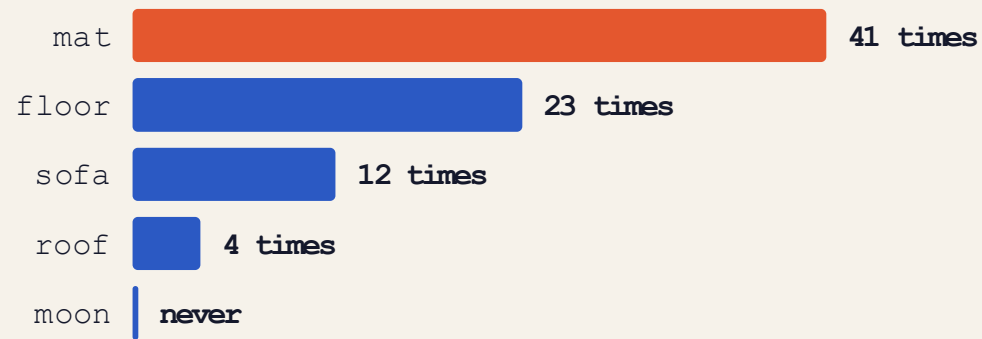


Text supplies its own answer key. Every word in every document is a free exercise: here is what came before, now what comes next?

No human labels are needed. This is why training on enormous amounts of text is practical.

# The oldest approach: counting

TOY TALLY · WHAT FOLLOWED " sat on the" IN A PILE OF TEXT



Count what followed each short phrase, then predict whatever was most common.

These tallies are called **n-gram models**. For decades they powered spell-checkers and early predictive keyboards.

They are fast and simple, and they know nothing beyond the few words directly behind the gap.

# Where counting breaks down

## PROBLEM 1

### Unseen phrases score nothing

If “sat on the trampoline” never appeared in the pile of text, the tally has no opinion, even though any reader finds it sensible.

## PROBLEM 2

### The table explodes

Looking back two words needs a modest table. Looking back twenty needs more rows than there is text in the world to fill.

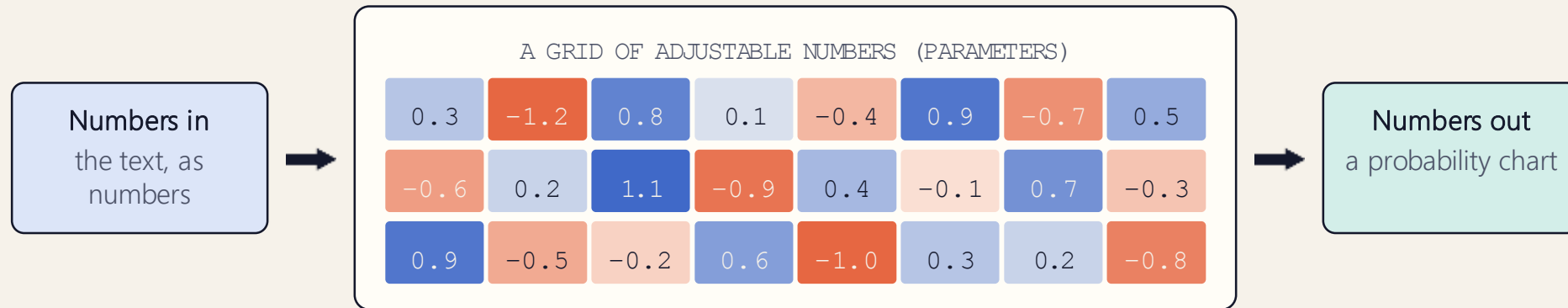
## PROBLEM 3

### No sense of similarity

“cat” and “kitten” are separate rows. Nothing learned about one transfers to the other.

A better predictor has to generalise: treat similar words alike and use context far beyond the last few words. That calls for a model that learns rather than counts.

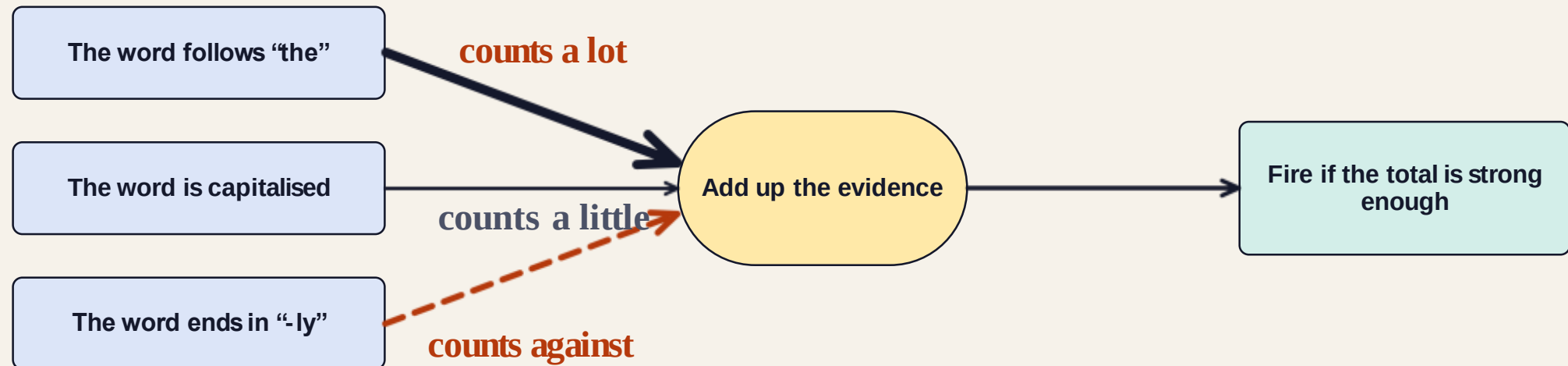
# A neural network in one picture



A neural network is a long recipe of simple arithmetic whose ingredients, the parameters, can be tuned.

Think of every cell as a dial. Small models have millions of dials; the largest language models have hundreds of billions. Training is the process of finding good settings for all of them.

# A single neuron: a tiny voter



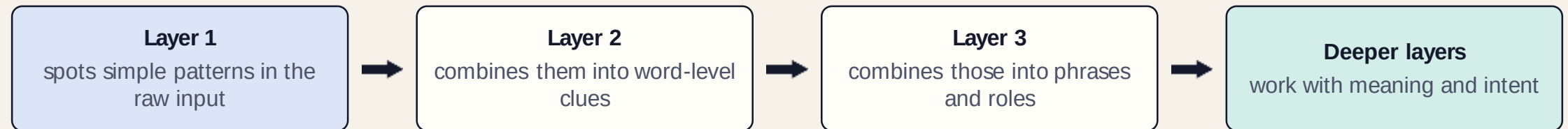
One neuron asks a narrow question. Here: is this word a noun?

Each input is a piece of evidence. The neuron weighs it by importance (line thickness), totals it, and passes on a signal only if the total is convincing.

**The importance of each a dial. Learning means adjusting adjusting these.**



# Layers: features built on features



Thousands of neurons sit side by side in a layer, and layers are stacked. Each layer only sees the output of the one before it.

This tidy ladder is a simplification, but the direction is right: later layers handle more abstract ideas.

# Learning as dial-turning



No one sets the dials by hand. The network starts with random settings and improves through a loop of guessing and correcting.

Part 9 opens up this loop. Until then, assume the dials have already been tuned and ask: what does the tuned machine do with a sentence?

# Part 1 in six ideas

01

## One job

A language model predicts the next token from the tokens so far.

02

## A chart, not a word

Its output is a share of 100% for every token it knows.

03

## Knowledge is a side effect

Guessing well across all text forces grammar, facts and reasoning patterns to form.

04

## Generation is a loop

Pick a token, append it, run again. Long answers are many passes.

05

## Counting falls short

Tallies cannot generalise or look far back. Learning is needed.

06

## Dials, tuned by practice

A neural network is arithmetic with adjustable parameters, set by guess-and-correct.

## PART 02

# Before transformers

Earlier language models read text one word at a time. Seeing where they struggled makes the transformer's design choices feel obvious.

## IN THIS PART

Order matters

Recurrent networks

Fading memory

Encoder and decoder

The first attention

The 2017 paper

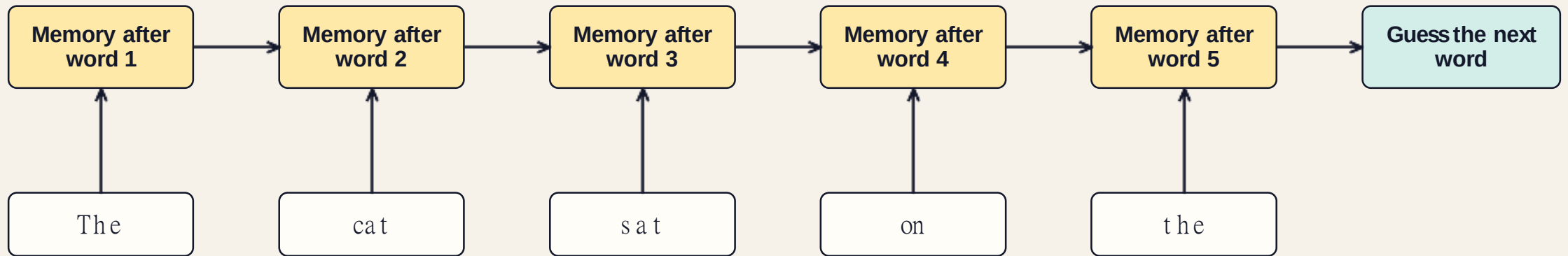
# Text as an ordered sequence



Same three words, opposite meaning. A language model has to handle both which words appear and the order they appear in.

Sentences also vary in length, from two words to thousands. Any design has to cope with a sequence of any size.

## Recurrent networks: one word at a time

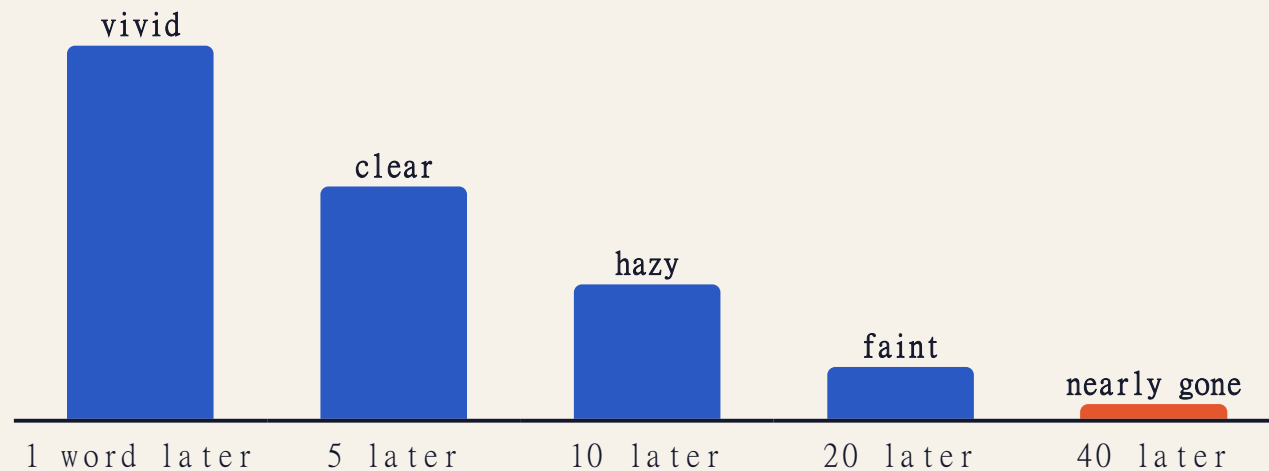


A recurrent network (RNN) reads left to right. After each word it rewrites a single memory note and hands it to the next step.

Everything the model knows about the sentence so far must fit in that one note.

# The fading memory problem

HOW WELL THE FIRST WORD IS REMEMBERED · ILLUSTRATIVE



Each rewrite of the memory note blurs what was there before.

It is the telephone game: a message whispered through forty people rarely arrives intact. A fact from the start of a paragraph is often lost by the end.

# LSTMs: gates that guard the memory

## FORGET GATE

### What can be dropped?

Decides which parts of the old memory are no longer needed and can be wiped.

## INPUT GATE

### What is worth keeping?

Decides how much of the new word should be written into memory.

## OUTPUT GATE

### What matters right now?

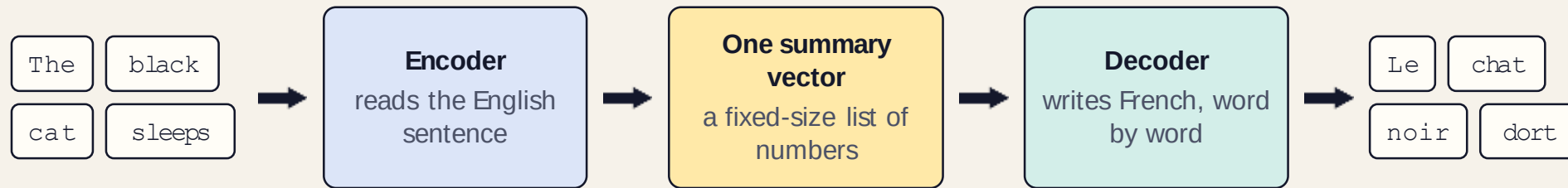
Decides which parts of memory to reveal for the current prediction.

The Long Short-Term Memory network (1997) added learned gates so that important information could survive many steps.

LSTMs dominated language tasks for years. They stretched memory further, but still read strictly one word at a time and still struggled with very long text.



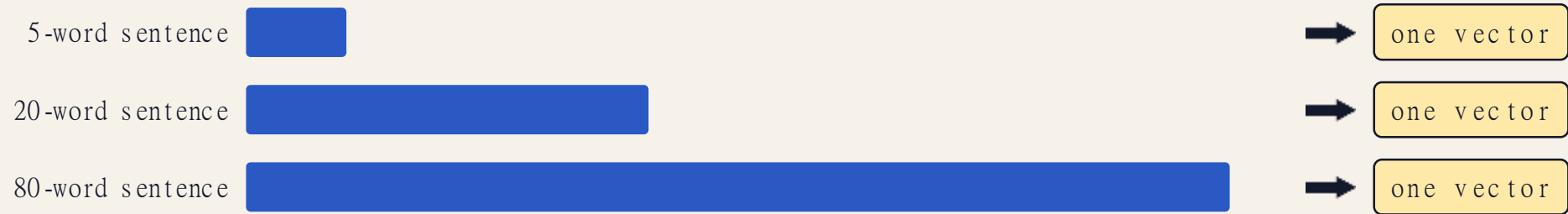
# The translation setup: encoder and decoder



In 2014, sequence-to-sequence models paired two recurrent networks: one to read the source sentence, one to write the translation.

The only link between them was the encoder's final memory note.

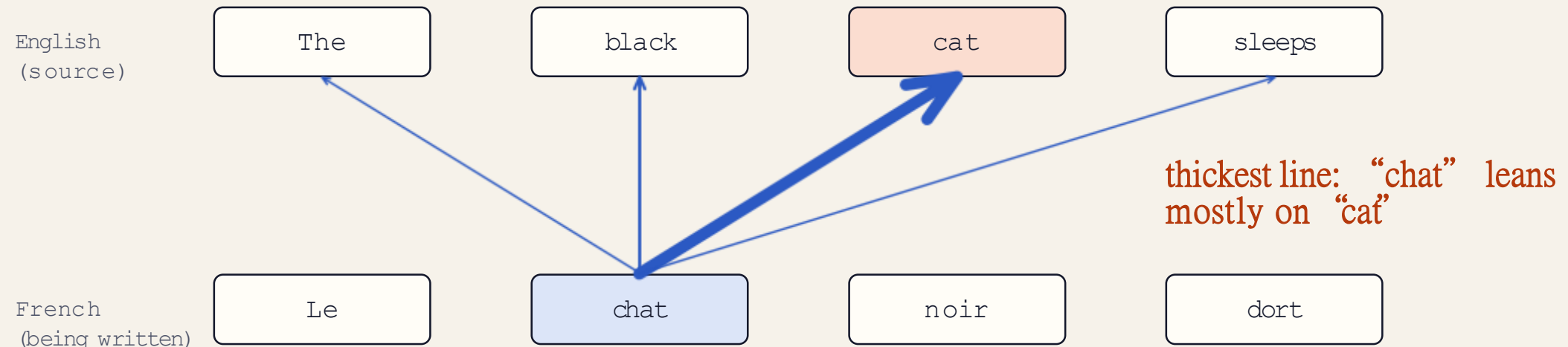
# The single-vector bottleneck



However long the input, the decoder received the same small summary. Short sentences translated well; long ones lost detail.

**Imagine translating a whole paragraph from a single sticky note about it.**

## Attention arrives (2014)



Instead of relying on one summary, let the decoder look back at every source word each time it writes a word, and decide how much to use each one.

This look-back with adjustable weights was named **attention**. It removed the bottleneck and sharply improved translation of long sentences.

# An alignment chart

ROWS: FRENCH WORD BEING WRITTEN · COLUMNS: ENGLISH WORD LOOKED AT

| The | black | cat | sleeps |      |
|-----|-------|-----|--------|------|
|     |       |     |        | Le   |
| 88% | 4%    | 6%  | 2%     | chat |
| 5%  | 7%    | 85% | 3%     | noir |
| 3%  | 86%   | 8%  | 3%     | dort |
| 2%  | 3%    | 5%  | 90%    |      |

Dark cells show where the decoder looked.

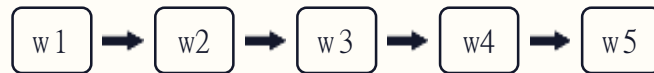
French puts the adjective after the noun, so “chat” looks at “cat” and “noir” looks at “black”. The model discovered this crossing on its own.

**Each row adds up to 100%. Illustrative values.**

# Two problems that remained

## PROBLEM 1 · SPEED

### Reading in single file



Word five cannot be processed until words one to four are done. Graphics processors excel at doing many things at once, and this design gave them little to do in parallel. Training was slow.

## PROBLEM 2 · DISTANCE

### Long chains between distant words



Even with attention bolted on, the core of the model still passed information step by step. Linking a word to another far away meant surviving every step in between.

Attention was clearly the most useful part. That raised a bold question: what if attention were the whole model?

# The 2017 proposal

PAPER TITLE

## Attention Is All You Need

Vaswani and seven co-authors, Google, 2017. Written to improve machine translation.

REMOVED

Recurrence is dropped entirely

No memory state is passed from word to word.

KEPT

Attention does the communicating

Every word can look directly at every other word.

RESULT

The Transformer

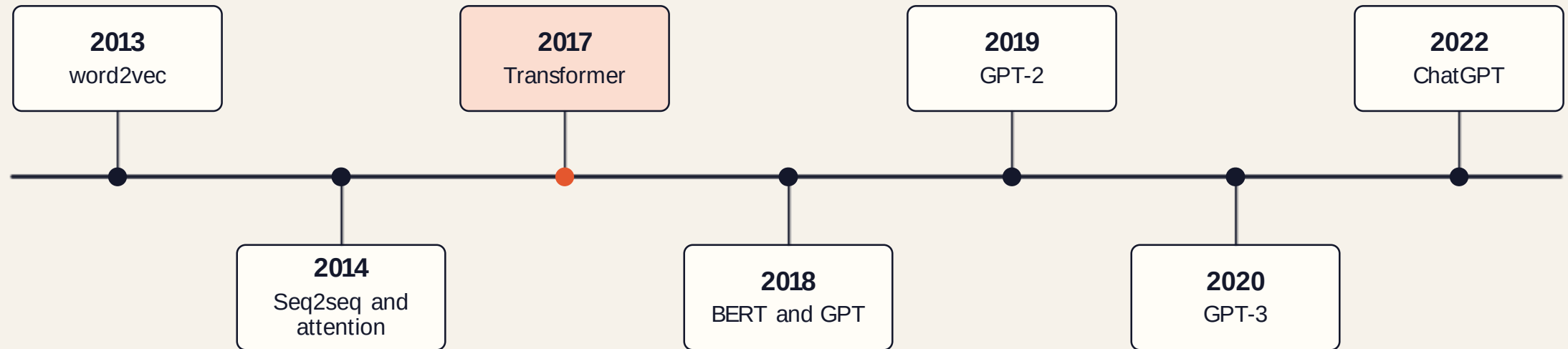
The architecture behind today's large language models.

# What the transformer changed

|                                | Recurrent network                   | Transformer                                    |
|--------------------------------|-------------------------------------|------------------------------------------------|
| How it reads                   | One word at a time, in order        | All words at once                              |
| Path between two distant words | Many steps, one per word in between | A single step, through attention               |
| Use of parallel hardware       | Poor: each step waits for the last  | Excellent: all positions computed together     |
| Word order                     | Built into the reading process      | Must be added as extra information             |
| Cost as text gets longer       | Grows in step with length           | Grows much faster: every word meets every word |

The last two rows are the price paid. Parts 5 and 6 explain both.

# A short timeline



Within five years of the 2017 paper, transformer-based language models had moved from research labs to hundreds of millions of users.



## Part 2 in six ideas

01

### Order and length

Language is a sequence of varying length; both content and order matter.

02

### RNNs

Read one word at a time and carry a single memory note forward.

03

### Fading memory

Each step blurs older information. LSTM gates helped but did not cure it.

04

### The bottleneck

Encoder–decoder models squeezed a whole sentence into one vector.

05

### Attention

Let the model look back at every word with adjustable weights.

06

### The transformer

Drops recurrence and keeps only attention: parallel, and one step between any two words.

# Tokens

A neural network cannot read letters. Before anything else, text is chopped into pieces called tokens and each piece is given a number.

## IN THIS PART

Characters, words, subwords

Byte-pair encoding

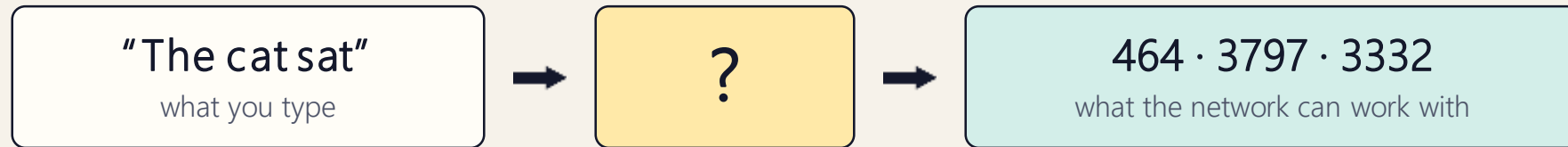
The vocabulary

Token IDs

Special tokens

Token quirks

# Only numbers allowed



A network only multiplies and adds numbers. So the first step is always the same: cut the text into pieces and replace each piece with a number.

The interesting question is what the pieces should be. There are three obvious candidates.

# Option A: one piece per character

"The cat sat unbelievably still"

T h e \_ c a t \_ s a t \_ u n b e l i e v a b l y \_ s t i l l

## STRENGTH

### A tiny alphabet

A hundred or so symbols cover everything, including words never seen before.

## WEAKNESS

### Long sequences, little meaning

Five words became thirty pieces. A single letter says almost nothing, so the model must first relearn spelling before it can think about meaning.

Simple and complete, but it wastes the model's effort on assembling letters.

## Option B: one piece per word

“The cat sat unbelievably still”

The cat sat unbelievably still

### STRENGTH

#### Short and meaningful

Each piece is a unit people recognise, and sequences stay short.

### WEAKNESS

#### An endless dictionary

Every form needs its own entry: believe, believes, believable, unbelievably. New words, names and typos have no entry at all.

Natural for people, but the dictionary never stops growing and unknown words break it.

# Option C: subword pieces

"The cat sat unbelievably still"

The cat sat un believ ably still

## STRENGTH

Common words stay whole

Frequent words get one piece each. Rare words are built from reusable parts, so nothing is ever unknown.

## WEAKNESS

Pieces are not always tidy

Splits follow frequency in the training text, not grammar. Some pieces look odd to a human reader.

The middle path, and the one every modern language model takes. These pieces are called **tokens**.

# One phrase, three ways

CHARACTERS

29

pieces

T r a n s f o r m e r s \_ a r e \_ u n b e l i e v a b l e

SUBWORD TOKENS

6

pieces

Transform ers are un believ able

WORDS

3

pieces

Transformers are unbelievable

**Subword splits shown are illustrative; each tokenizer has its own.**

# Byte-pair encoding: learning the pieces

---

## 01 Start with single characters

Every word in the training text is spelled out letter by letter.

---

## 02 Count neighbouring pairs

Find the two pieces that sit next to each other most often.

---

## 03 Merge the winning pair

Glue them into one new piece and add it to the vocabulary.

---

## 04 Repeat until the vocabulary is full

Stop at a chosen size, often tens of thousands of pieces.

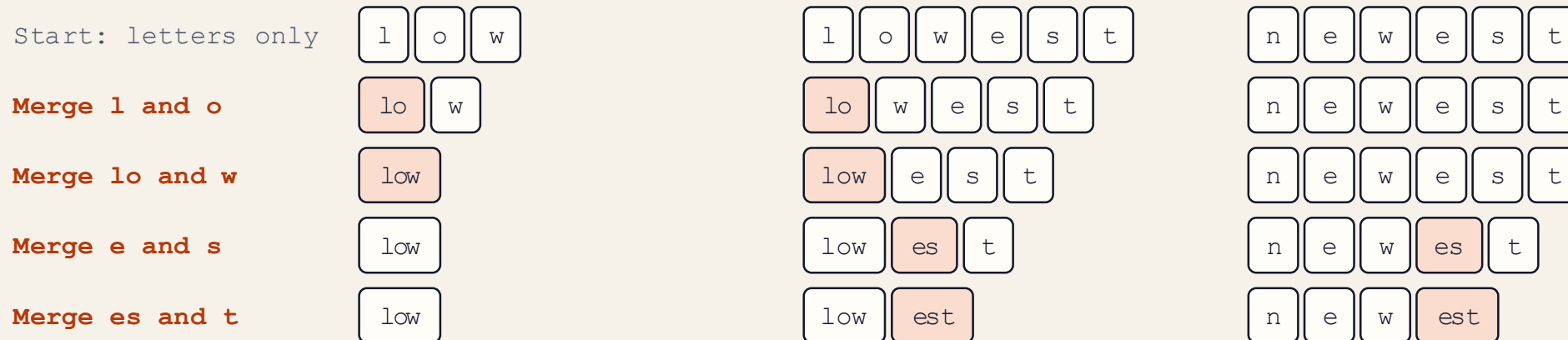
Nobody designs the token list. It is grown from data by one simple rule: merge what occurs together most.

Frequent words end up as single tokens. Rare words stay as several smaller ones.



# Byte-pair encoding in action

TOY TRAINING TEXT: " low" THREE TIMES · " lowest" TWICE · " newest" TWICE



After four merges the vocabulary has gained lo, low, es and est.

"lowest" is now two tokens: low and est. A word never seen in training, such as "slowest", can still be written with known pieces.

Each merge picks the pair that occurs most often at moment.

# The vocabulary: a numbered dictionary

| Token | ID   |
|-------|------|
| The   | 464  |
| cat   | 3797 |
| sat   | 3332 |
| on    | 319  |
| the   | 262  |
| mat   | 2603 |

When merging stops, the list of pieces is frozen and each piece gets a fixed number: its token ID.

GPT-2's vocabulary holds 50,257 tokens. Many newer models use vocabularies of a similar order: tens of thousands to a few hundred thousand.

The ID is just a position in the list. Token 3797 is not bigger or better than token 319.

**marks a space. IDs shown are examples in the style of GPT-2's vocabulary.**

# From text to token IDs

1 · THE TEXT

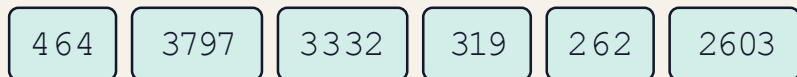
The cat sat on the mat



2 · SPLIT INTO TOKENS



3 · LOOK UP EACH ID



This list of whole numbers is the only thing the model receives. The program that does the splitting is called a **tokenizer**, and it sits outside the neural network.

Run it backwards to turn the model's output IDs into text again.

# Special tokens

**START****Beginning of text**

Marks where a document or conversation begins.

**STOP****End of text**

The model produces this token when it judges the reply complete. Software then stops the loop.

**ROLES****Who is speaking**

Chat models wrap each turn in markers for system, user and assistant so the model can tell them apart.

**PADDING****Filler**

During training, shorter examples are padded to a common length. The model is told to ignore these.

Besides pieces of words, the vocabulary reserves a few tokens that never appear in ordinary writing. They act as stage directions.

&lt;start&gt;

&lt;user&gt;

Hello

!

&lt;assistant&gt;

Hi

there

&lt;end&gt;

**a chat, as the model sees it**

# Token quirks worth knowing

| Quirk                   | Example                                                    | Consequence                                                      |
|-------------------------|------------------------------------------------------------|------------------------------------------------------------------|
| Spaces belong to tokens | “cat” and “_cat” are different tokens                      | The same word mid-sentence and at the start may be different IDs |
| Capitals matter         | “The” and “the” are different tokens                       | Each form is learned separately, then linked through training    |
| Rare words shatter      | A long chemical name may be ten tokens                     | Unusual vocabulary costs more and is harder for the model        |
| Numbers split in chunks | A long number becomes several pieces                       | Digits do not line up neatly the way they do on paper            |
| Languages differ        | Text in less-represented languages often needs more tokens | The same message can cost more in one language than another      |

# Behaviour that tokens explain

## SPELLING

### Counting letters in a word

The model sees “strawberry” as a couple of pieces, not ten letters. It has to recall the spelling rather than look at it.

## ARITHMETIC

### Sums with long numbers

Digits are grouped into uneven chunks, so place value is not laid out the way a pupil would write it.

## WORDPLAY

### Rhymes, anagrams, reversals

Tasks about the letters inside words are indirect for a system whose smallest unit is usually larger than a letter.

Many well-known stumbles of language models are not failures of intelligence. They follow from what the model can and cannot see.

# Token budgets

1,024

GPT-2 (2019)  
tokens of context

2,048

GPT-3 (2020)  
tokens of context at launch

Over 100,000

Many current models  
some accept a million tokens or more

A model can only consider a fixed number of tokens at once. This limit is its **context window**, and it is counted in tokens, not words or pages.

Rule of thumb for English: 100 tokens is roughly 75 words.

# Part 3 in six ideas

01

## Text becomes numbers

A tokenizer cuts text into pieces and replaces each with an ID.

02

## Subwords win

Common words stay whole; rare words are assembled from parts.

03

## BPE

The token list is grown by repeatedly merging the most frequent neighbouring pair.

04

## A fixed dictionary

Tens of thousands of tokens, each with a permanent ID.

05

## Special tokens

Reserved markers for start, stop, speaker roles and padding.

06

## Tokens shape behaviour

Costs, limits and several odd failures all trace back to tokens.



# Embeddings

A token ID is only a name tag. To compute with a token, the model swaps the ID for a long list of numbers that captures what the token means.

## IN THIS PART

IDs carry no meaning

Lists of numbers

The embedding table

Meaning as location

Directions

The “bank” problem

# Token IDs carry no meaning

|      |                         |
|------|-------------------------|
| 3796 | an unrelated token      |
| 3797 | <code>[CLS] cat</code>  |
| 3798 | another unrelated token |

Neighbouring IDs are strangers. The number says where a token sits in the list and nothing about what it means.

If the model did arithmetic on raw IDs it would treat token 3797 as almost the same as token 3798, and as much “more” than token 319. Both are nonsense.

**An ID is a locker number. What matters is what is inside the locker.**

# An embedding: a list of numbers



Each token is swapped for a long list of numbers, called its **embedding** or **vector**. From here on, the model works only with these lists.

## SIZE

### How long is the list?

GPT-2 small uses 768 numbers per token.  
Larger models use several thousand.

## MEANING

### What do the numbers mean?

No single number has a label. Together they place the token at a point in a space of meaning.

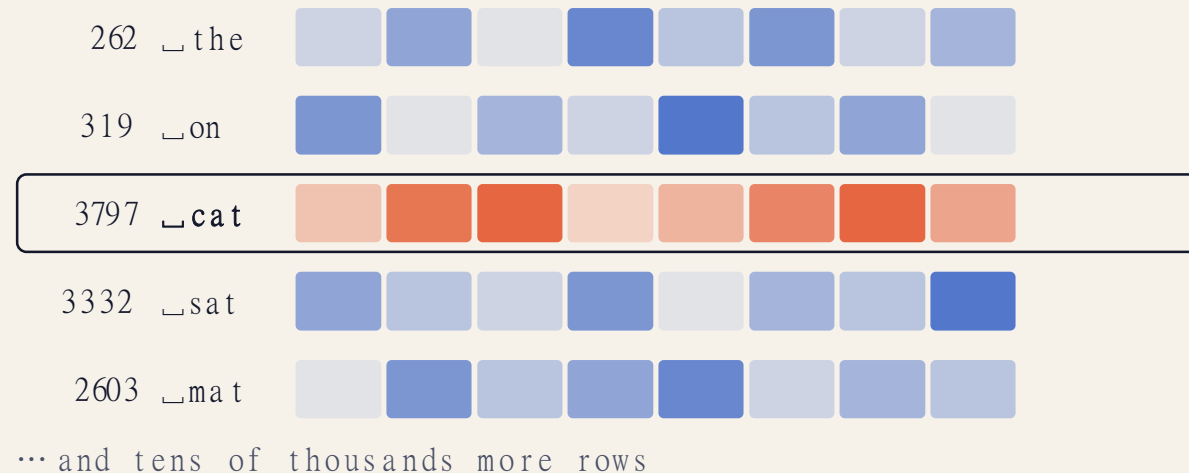
## LEGEND

### How to read the picture

Blue cells are positive, orange are negative, and darker means further from zero.

# The embedding table

ONE ROW PER TOKEN IN THE VOCABULARY · ONE COLUMN PER DIMENSION

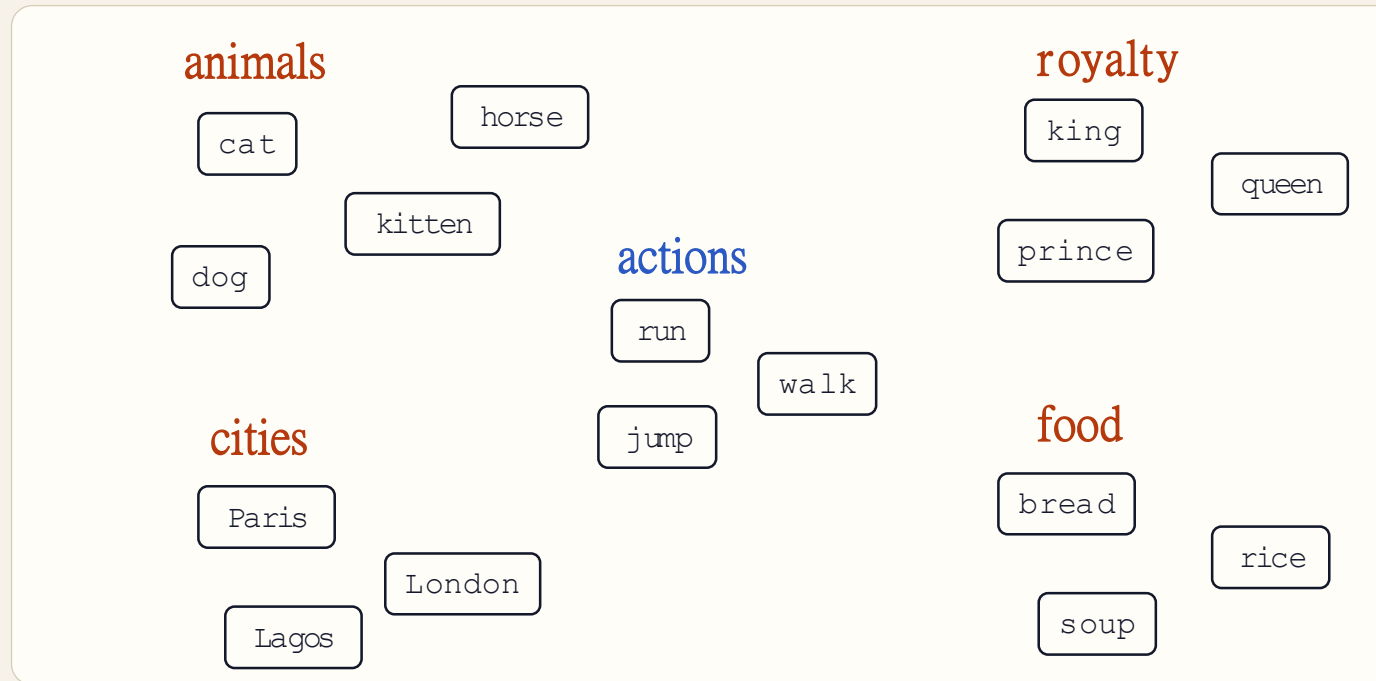


Turning an ID into a vector is a lookup: go to that row and copy it out.

For GPT-2 small the table has 50,257 rows and 768 columns: about 38.6 million numbers, every one of them learned during training.

Token 3797 arrives. Row 3797 is copied out. No calculation is involved.

# Meaning as a location on a map



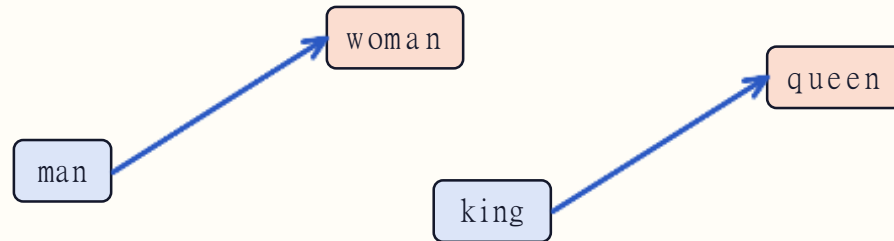
Picture every token as a pin on a map. Tokens used in similar ways sit close together.

Real embeddings live in hundreds or thousands of dimensions, not two. The idea is the same: nearby means similar.

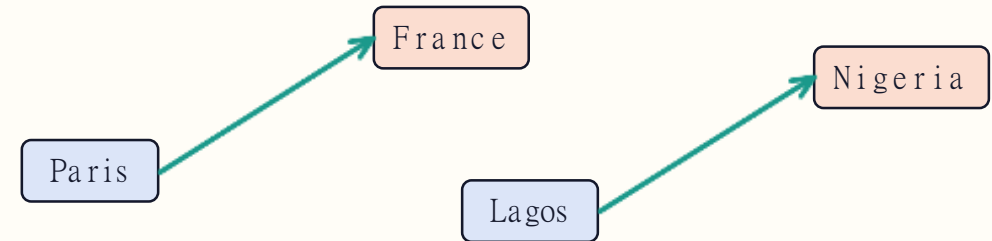
**Nobody drew this map. It forms by itself during training.**

# Directions carry meaning

one direction means “make it female”



another means “the country this city is in”



The step from “man” to “woman” is nearly the same step as from “king” to “queen”. Relationships between words show up as consistent directions on the map.

# Dimensions as hidden features

A CARTOON OF THREE EMBEDDINGS

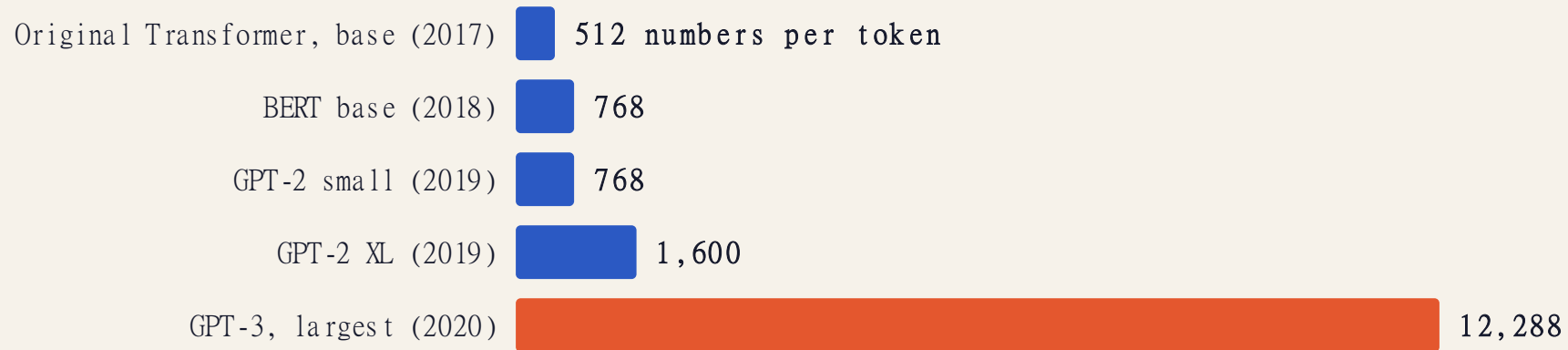
|       | living | royal | place | large |
|-------|--------|-------|-------|-------|
| cat   | high   | low   | low   | low   |
| king  | high   | high  | low   | some  |
| Paris | low    | low   | high  | high  |

It helps to imagine each dimension as a dial for some quality.

With enough dials, “cat”, “king” and “Paris” each get a distinct combination of settings.

**A cartoon only. Real dimensions have no labels, and each quality is smeared across many numbers at once.**

## How long the lists are



The length of each token's vector is the model's **width**. A wider model can keep more distinct qualities apart at the same time.

Width is one of three main size knobs. The others, depth and heads, come later.



# Similarity as shared direction

HOW ALIKE TWO TOKEN VECTORS ARE · 100 MEANS IDENTICAL · ILLUSTRATIVE

| cat | kitten | dog | car | Paris |        |
|-----|--------|-----|-----|-------|--------|
| 100 | 92     | 78  | 21  | 8     | cat    |
| 92  | 100    | 74  | 17  | 6     | kitten |
| 78  | 74     | 100 | 24  | 9     | dog    |
| 21  | 17     | 24  | 100 | 15    | car    |
| 8   | 6      | 9   | 15  | 100   | Paris  |

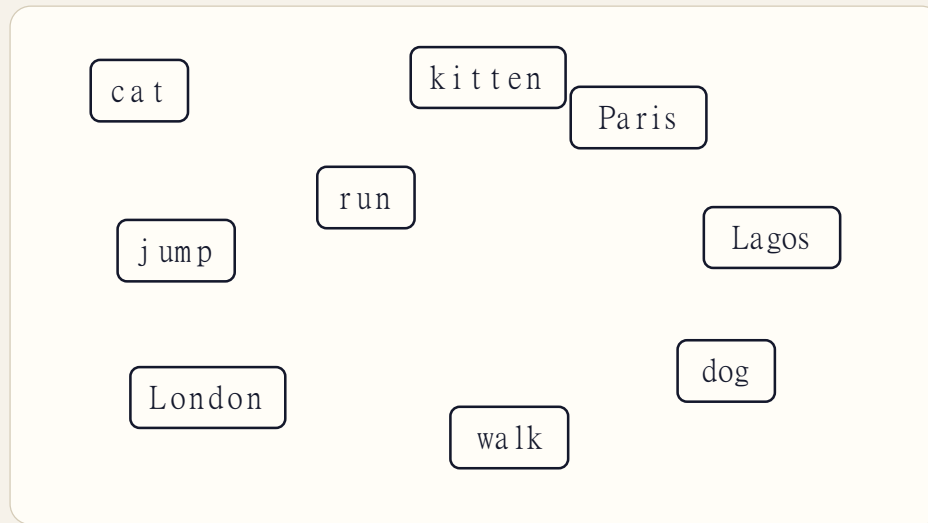
Two vectors are similar when they point in nearly the same direction.

This single test, how well two arrows line up, is used everywhere in a transformer. Attention is built on it.

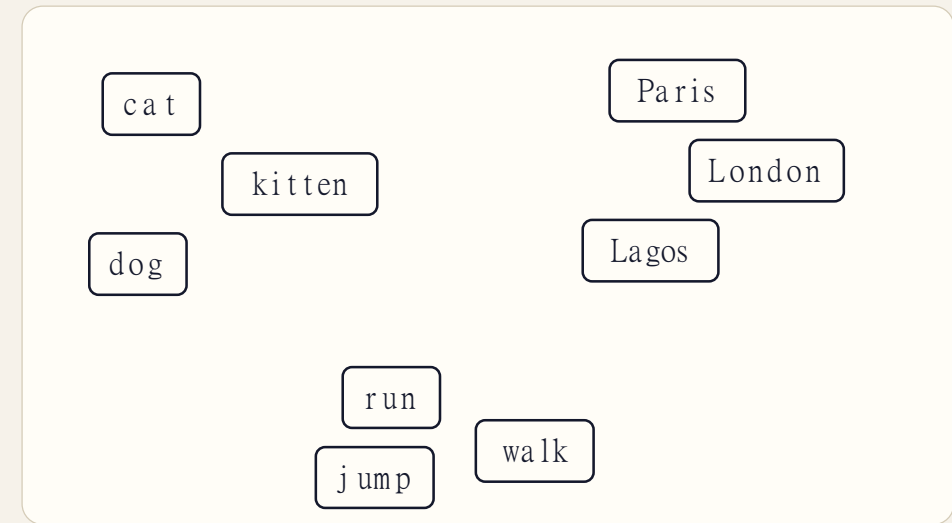
**Remember this test. In Part 6 it decides which words listen to which.**

# Learned, not hand-written

BEFORE TRAINING · RANDOM PLACEMENT

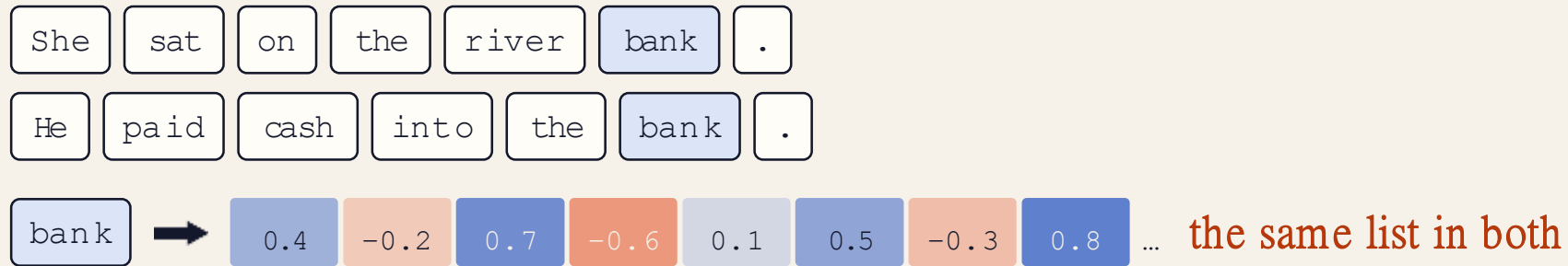


AFTER TRAINING · SIMILAR TOKENS CLUSTER



Embeddings begin as random numbers. Each training step nudges them, and tokens that appear in similar surroundings drift together.

# One vector per token: the “bank” problem



The lookup table has one row for “bank”. Riverside or high street, the starting vector is identical. It cannot yet know which meaning is intended.

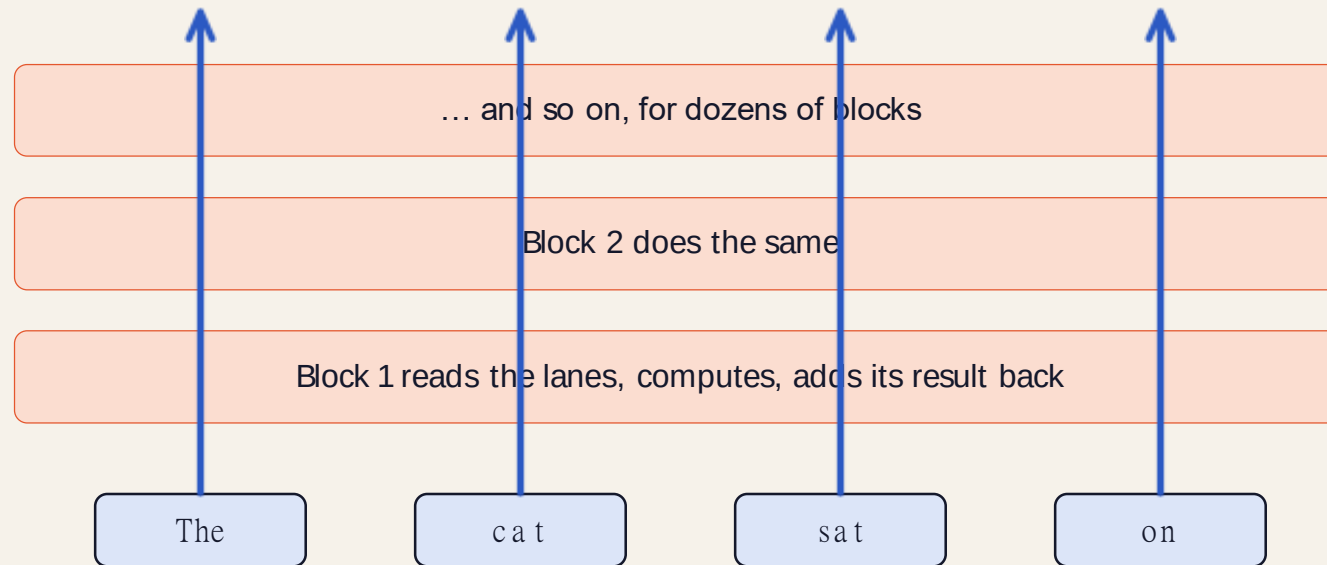
The same is true of “it”, “run”, “light”, “Apple” and thousands of other tokens whose meaning depends on their neighbours.

# The transformer's job, restated



Context-free vectors go in. Context-aware vectors come out. Every part of the architecture exists to make that conversion.

# A running notebook for every token



Each token keeps its own lane of numbers running up through the model.

A block never replaces a token's vector. It reads what is there and adds a small update. Researchers call these lanes the **residual stream**.

Think of a notebook passed up a line of of editors. Each adds notes; nobody pages out.

# Part 4 in six ideas

01

## IDs are name tags

The number itself means nothing and cannot be computed with.

02

## Embeddings

Each token is swapped for a long list of learned numbers.

03

## A lookup table

One row per token; converting an ID is copying a row.

04

## Meaning is geometry

Similar tokens sit close; relationships appear as directions.

05

## Context-free start

“bank” begins with the same vector in every sentence.

06

## The goal

Turn context-free vectors into context-aware ones, by adding updates lane by lane.

# Position

Attention, the heart of the transformer, has no built-in sense of word order. Position has to be written into the vectors on purpose.

## IN THIS PART

Order blindness

Position stamps

A learned table

Waves and clocks

Relative and rotary

The input, assembled

# Attention is order-blind

ORIGINAL

the dog chased the cat

SHUFFLED

cat the the chased dog

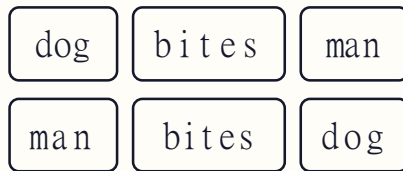
As Part 6 will show, attention compares every token with every other token. It treats the input as a bag of tokens, with no idea which came first.

**Without help, these two inputs would look identical to the model.**



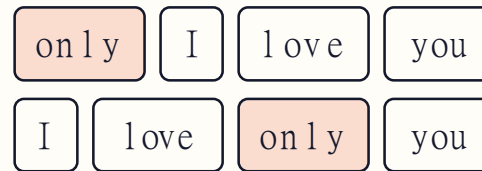
# Why order cannot be ignored

## ROLES



Who did what to whom depends entirely on position.

## SCOPE



Moving one word changes what it applies to.

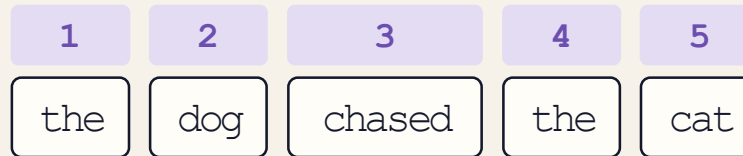
## NEGATION



Negation flips whatever follows it.

A recurrent network got order for free by reading in sequence. A transformer reads everything at once, so order must be supplied another way.

# The fix: stamp each token with its position



Alongside its meaning, each token's vector also carries a mark saying where in the sequence it sits.

## EFFECT

### Two copies of “the”

They share a meaning vector, but their stamps differ, so the model can tell first from fourth.

## FORM

### Stamps are vectors too

A position is written as a list of numbers the same length as a token vector, so the two can be combined.

# A learned position table

ONE ROW PER POSITION

|            |  |  |  |  |  |  |  |
|------------|--|--|--|--|--|--|--|
| position 1 |  |  |  |  |  |  |  |
| position 2 |  |  |  |  |  |  |  |
| position 3 |  |  |  |  |  |  |  |
| position 4 |  |  |  |  |  |  |  |

... one row for every position allowed

The simplest scheme is a second lookup table: row one for the first token, row two for the second, and so on.

Like token embeddings, these rows start random and are learned. GPT-2 works this way, with 1,024 rows, which is why it cannot read more than 1,024 tokens.

Token table: what is this? Position table: where is it?

# Sinusoidal positions: a clock with many hands

## FAST HAND

### The second hand

Sweeps round quickly. It separates moments that are close together, and repeats often.

## MEDIUM HAND

### The minute hand

Moves more slowly. It separates moments that the second hand confuses.

## SLOW HAND

### The hour hand

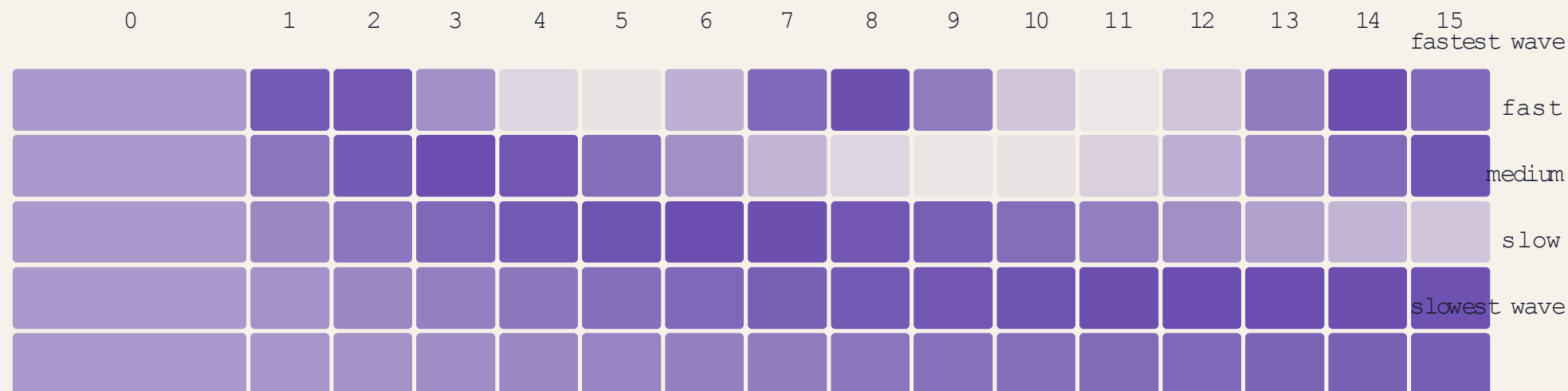
Barely moves. It tells apart times that are far from each other.

The original Transformer wrote position with a set of waves, some fast and some slow. Reading all of them together pins down any position, just as three hands pin down a time.

**Nothing is learned here. The pattern is fixed by a rule, so it extends to any length.**

# Position fingerprints

COLUMNS: POSITIONS 0 TO 15 · ROWS: FIVE WAVES OF DIFFERENT SPEED · DARKER MEANS HIGHER

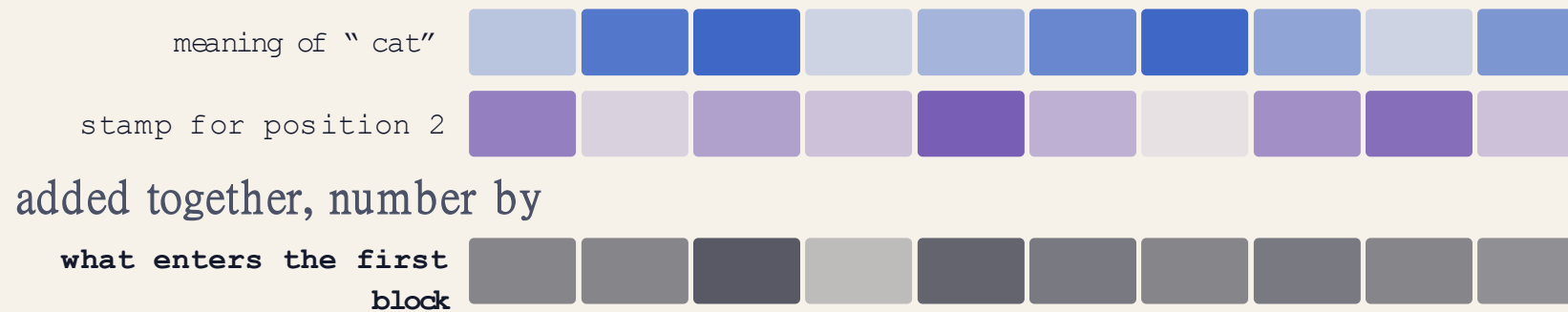


Read any column top to bottom: that pattern of light and dark is the position's fingerprint.

No two columns match. Nearby positions have similar fingerprints, and distant ones differ more.

**Top rows flicker quickly.  
rows change slowly, like the hour  
hour hand.**

# Combining meaning and position



The two lists are simply added. One vector now says both “this is cat” and “this is the second token”.

It sounds as if adding would scramble the two messages. In a space with hundreds of dimensions there is room for both, and the model learns to read each one back out.

# Relative positions and rotation

## ABSOLUTE

### “I am token 57”

Each token knows its own index. Simple, but the model must work out distances for itself, and long inputs go beyond anything it practised on.

## RELATIVE

### “You are three tokens before me”

What usually matters in language is the gap between two words, not their page coordinates. Relative schemes encode the gap directly.

## ROTARY (2021)

### “Turn each vector by its position”

Each token's vector is rotated by an angle that grows with its position. When two tokens are compared, only the difference between their angles, their distance apart, shows up.

Most current large language models use rotary position encoding, applied inside attention rather than added at the input.

# Position and the length limit

## LEARNED TABLE

### A learned table has a last row

With 1,024 rows there is no stamp for token 1,025. The limit is hard-wired.

## WAVES AND ROTATION

### A rule can be extended

Waves and rotations are defined for any position, so nothing breaks outright on longer input.

## IN PRACTICE

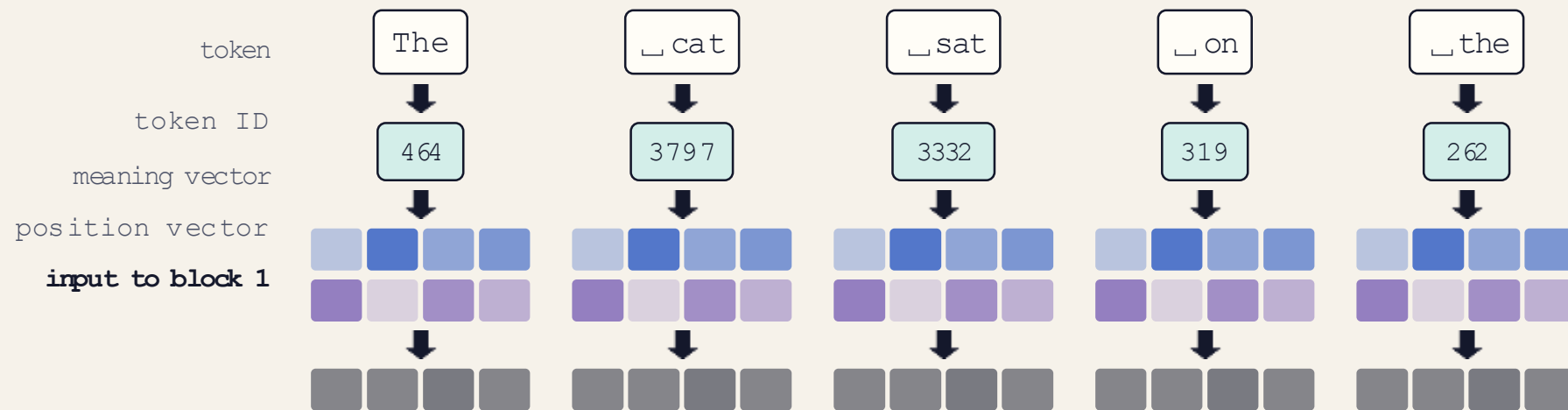
### Practice still matters

A model that only ever trained on short texts handles long ones poorly, whatever the encoding allows.

How position is written is one of the things that decides how much text a model can take in. The other is cost, which Part 6 explains.



# The input pipeline, assembled



Five tokens in, five vectors out, each carrying meaning and position. This grid of numbers is what the first transformer block receives.

# Part 5 in six ideas

01

## Order-blind by nature

Attention sees a bag of tokens unless position is supplied.

02

## Position stamps

Each token's vector also encodes where it sits.

03

## Learned table

A row per position; simple, with a hard length limit.

04

## Waves

A fixed pattern of fast and slow waves gives every position a unique fingerprint.

05

## Rotary encoding

Rotating vectors by position makes distance, not index, what attention sees.

06

## Input ready

Meaning plus position, one vector per token, enters the first block.

# Attention

The mechanism that lets every token collect information from the others. This is the centre of the architecture, so we take it slowly.

## IN THIS PART

Which words matter?

Query, key, value

Five steps

Heatmaps

The causal mask

Many heads

# The question attention answers

For each token: which other tokens help me understand what I mean here, and how much should I take from each?

A PRONOUN

**“it” needs its noun**

Until it finds “the animal”, the word “it” could stand for anything.

A VERB

**“sat” needs its subject**

Who sat? The answer is a few tokens back, in “cat”.

AN AMBIGUOUS WORD

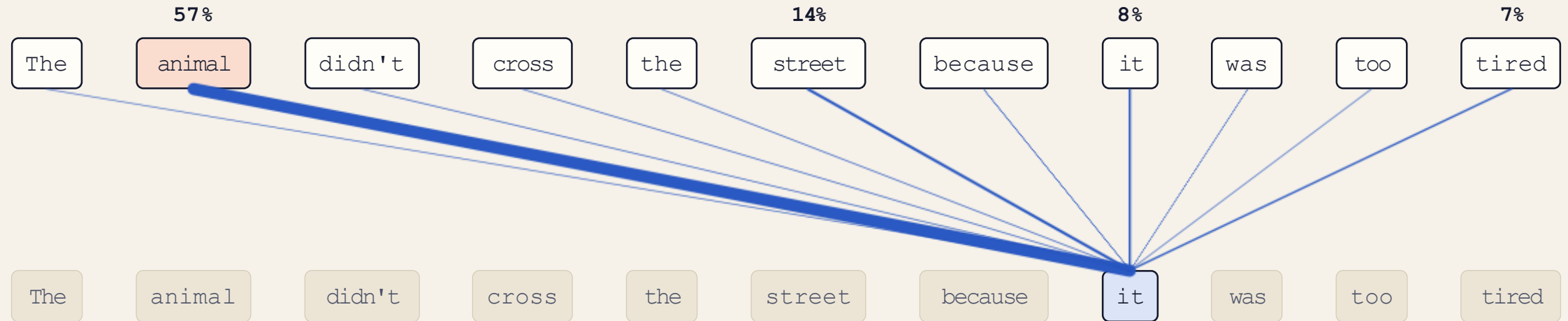
**“bank” needs a clue**

A nearby “river” or “cash” settles which meaning applies.

**Attention is how context reaches a token. It is the cure for the “ bank” problem.**

# A pronoun puzzle

TOKENS BEING LOOKED AT



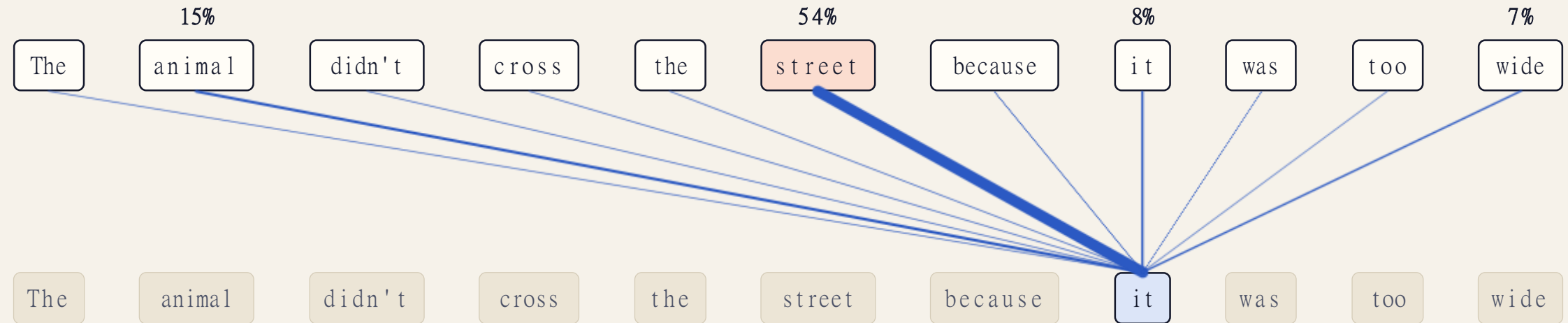
TOKEN DOING THE LOOKING

Asked what “it” refers to, a reader looks back and settles on “animal”, because animals get tired and streets do not.

A trained attention head does something very similar. The thickness of each line shows how much of its attention “it” gives to each token.

# One word changes the answer

TOKENS BEING LOOKED AT



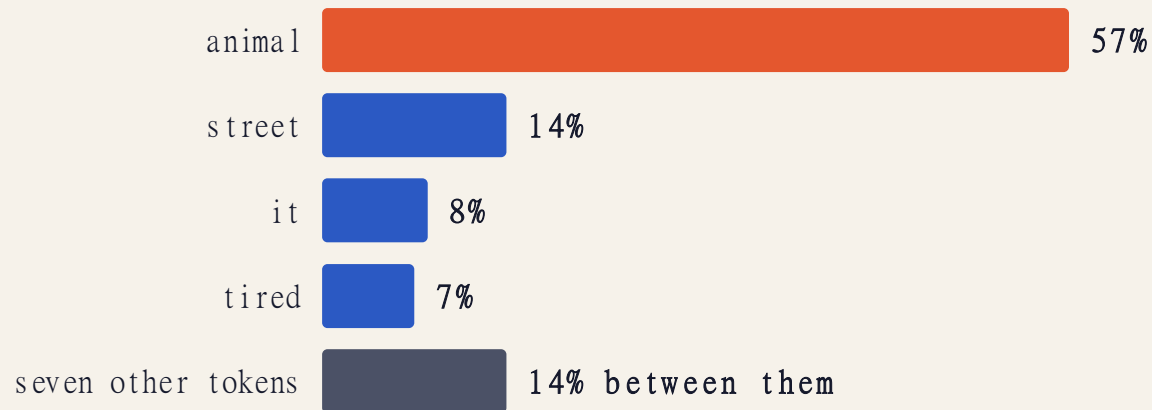
TOKEN DOING THE LOOKING

Swap “tired” for “wide” and the sensible reading flips: streets are wide, animals are not. The attention pattern flips with it.

Nothing was reprogrammed. The pattern is recomputed from the tokens every time, which is what makes it sensitive to context.

# Attention as a budget of 100%

HOW “it” SPENDS ITS ATTENTION · ILLUSTRATIVE



Every token has exactly 100% of attention to hand out.

Giving more to one token means giving less to the rest. The split is the token's answer to “who matters to me right now?”

**These shares are called attention weights.**

# Attention as a conversation

1 · ASK

Every token poses a question

“Here is the kind of information I am missing.”

2 · ADVERTISE

Every token holds up a sign

“Here is the kind of information I can provide.”

3 · DELIVER

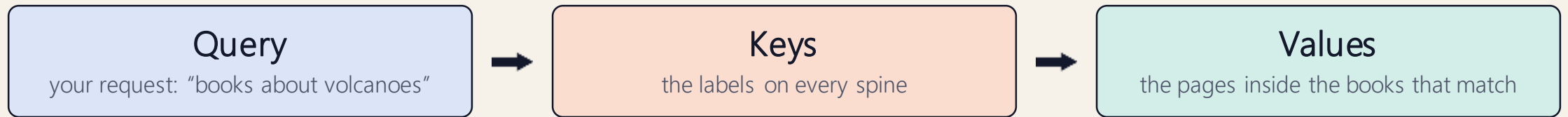
Good matches hand over content

Tokens whose signs fit the question pass their information to the asker, in proportion to the fit.

All tokens ask, advertise and deliver at the same moment. The three roles have names: **query**, **key** and **value**.



# The library analogy



You compare your request against every spine label, then read mostly from the best matches and skim a little from the near misses.

**Unlike a real library search, you never take just one book. You take a weighted blend of all of them.**

# The query: what am I looking for?

## Query

A token's query describes the information it wants from the rest of the sentence.

The animal didn't cross the street because it  
was too tired

it

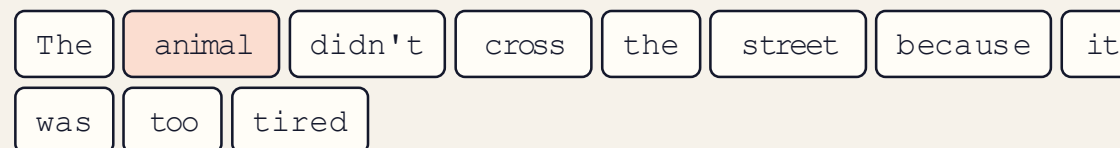
“I am a pronoun. I need a noun from earlier that I could be standing for.”

Each token produces its own query. A verb might ask for its subject; an adjective for the noun it describes.

# The key: what can I offer?

## Key

A token's key describes what kind of information it holds, so that queries can find it.



animal “ I am a noun. Singular: A living thing.”

Keys exist only to be matched against. “street” would advertise: noun, singular, a place.

## The value: what I hand over

# Value

A token's value is the content it actually passes along when a query selects it.

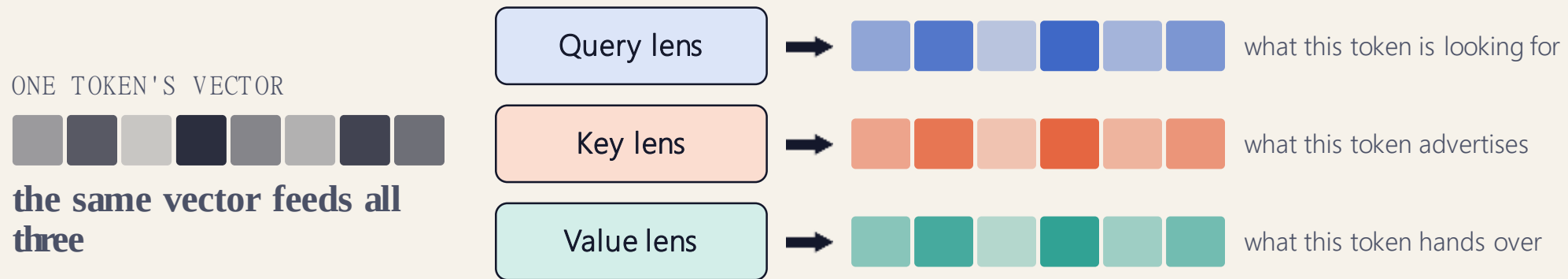


animal

“Take this: a creature, able to move, able to get tired.”

Key and value differ on purpose. The key is the label on the box; the value is what is inside.

## Three lenses on one vector

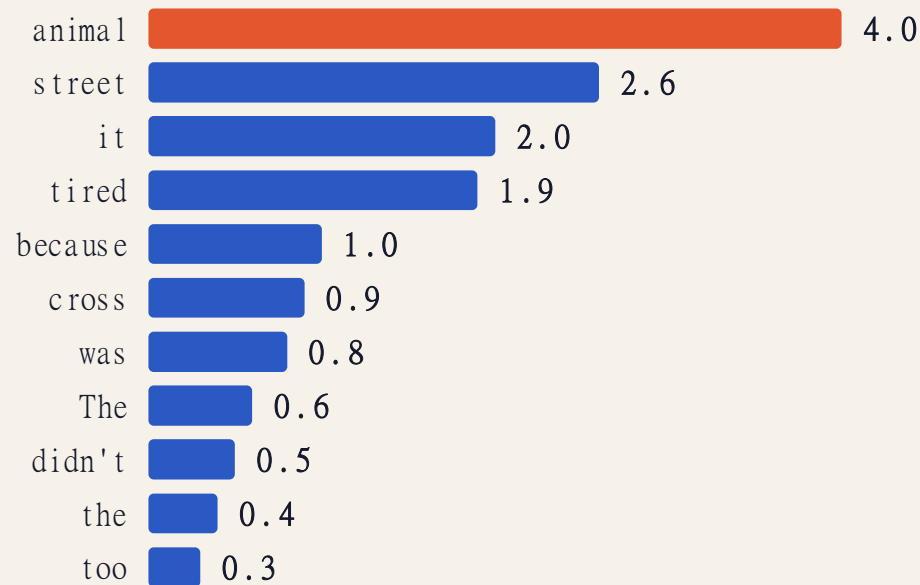


Queries, keys and values are not written in words. Each is a new list of numbers, made by passing the token's vector through a learned grid of numbers: a lens.

One attention head owns one set of three lenses and uses them for every token. What the lenses look for is learned in training.

# Step 1: score every pair

RAW MATCH SCORE: QUERY OF “it” AGAINST EACH KEY · ILLUSTRATIVE

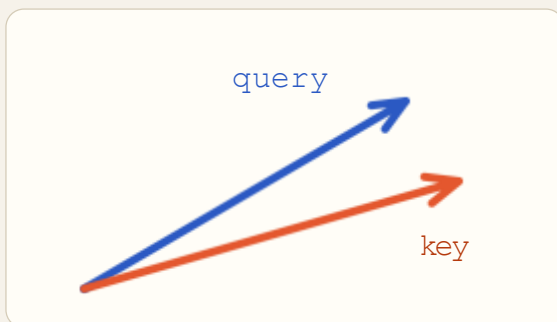


The query of “it” is compared with the key of every token, itself included.

Each comparison yields one number. A high score means: this key is what the query was asking for.

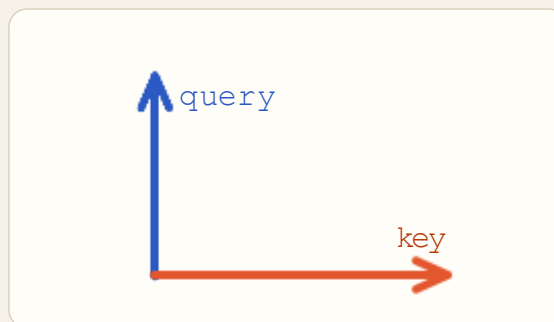
Eleven tokens, so eleven scores for “it” . Every other token gets its own eleven.

## How a match is measured



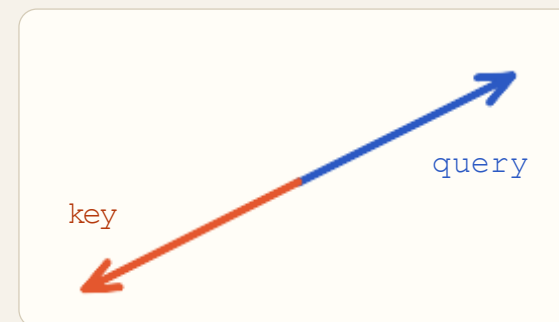
**Same direction: high score**

The key offers what the query wants.



**At right angles: near zero**

The key is irrelevant to this query.

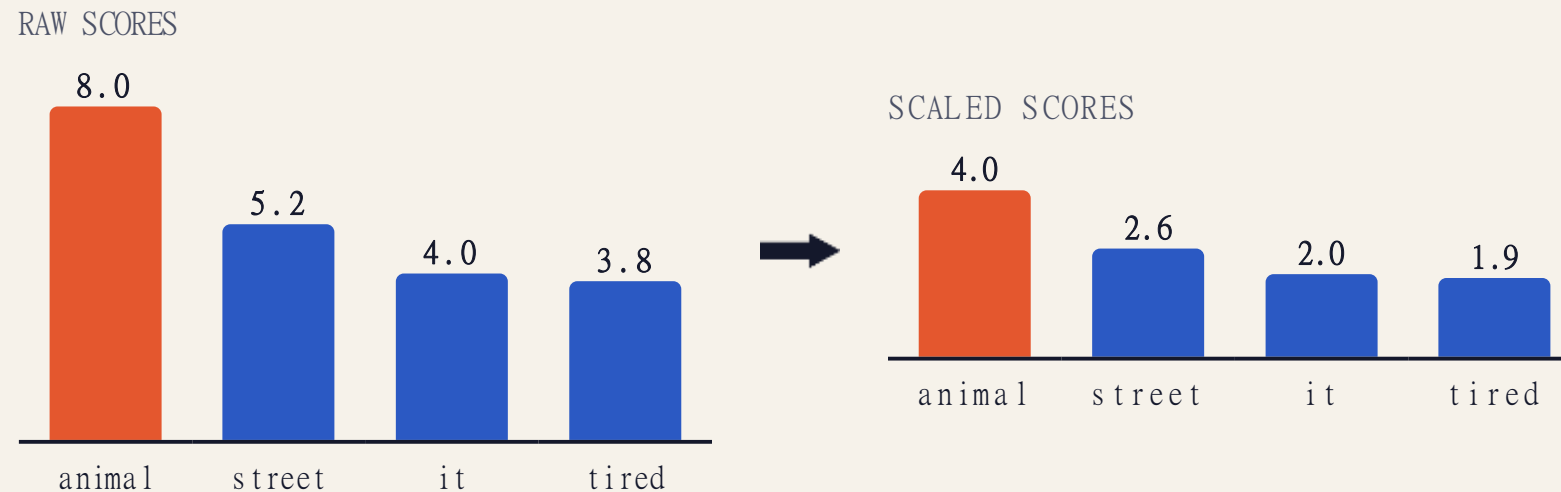


**Opposite: negative score**

The key is the reverse of what is wanted.

It is the similarity test from Part 4. Query and key are both arrows in the same space, and the score measures how well they line up.

## Step 2: scale the scores down



Long vectors produce large scores simply because they have more numbers in them.

So every score is shrunk by a fixed amount that depends only on the vector's length. The ranking is untouched; only the spread is reduced.

**Why bother? The next step overreacts to large gaps.**



## Step 3: turn scores into shares

### MOVE 1

#### Make every score positive

Each score is passed through a curve that maps any number, even a negative one, to a positive amount.

### MOVE 2

#### Let big scores grow fastest

The curve is steep: a slightly higher score becomes a much larger amount. Leaders pull ahead.

### MOVE 3

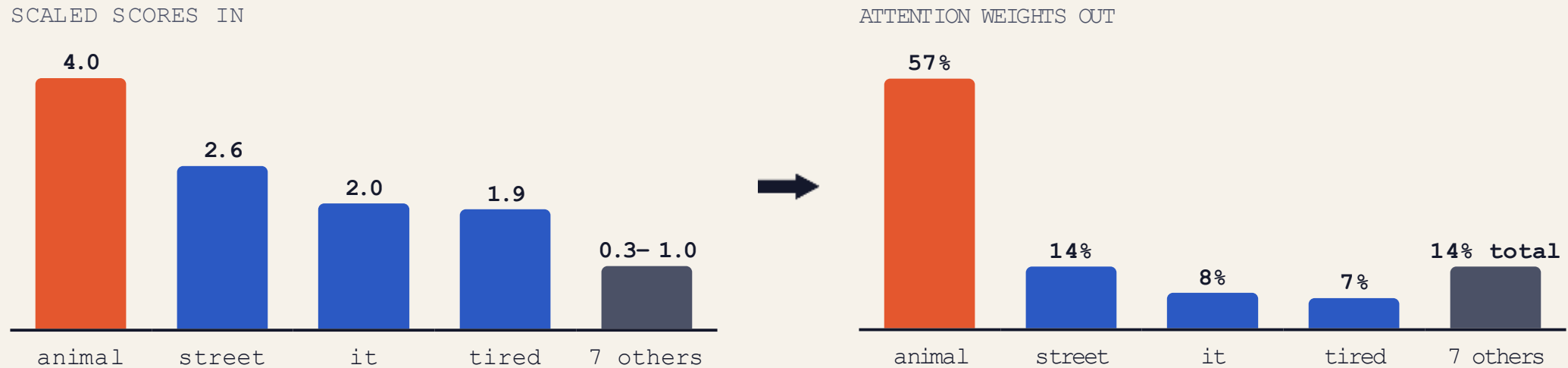
#### Share out 100%

Each amount is expressed as a fraction of the total, so the results always add up to exactly 100%.

This three-move recipe is called **softmax**. It turns any list of scores into a clean set of percentages, and it appears twice in every language model.

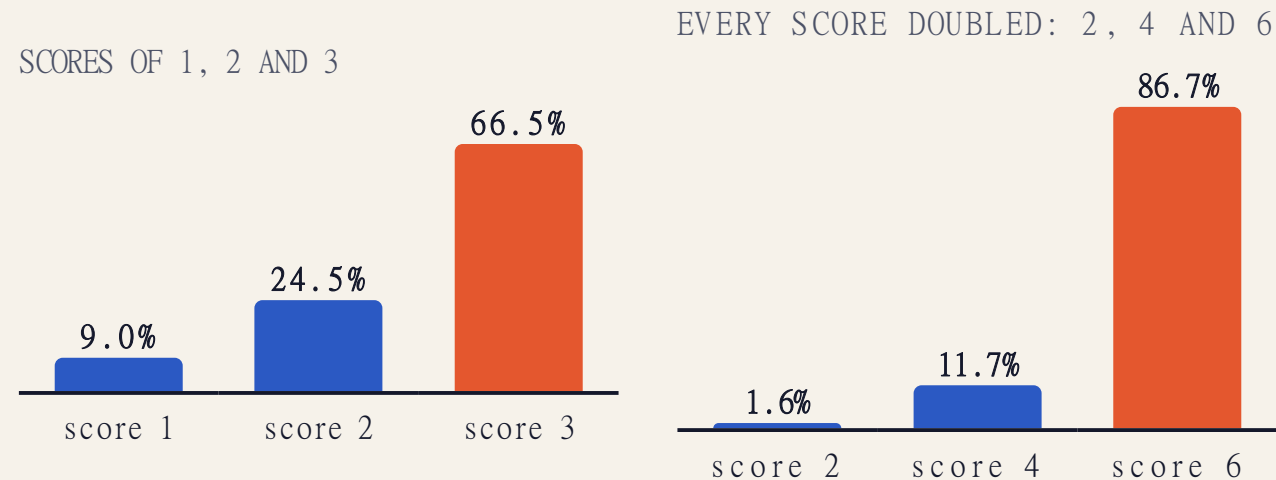
Here it produces attention weights. In Part 8 it produces the final word probabilities.

## Softmax applied to the scores of “it”



“animal” led by a modest margin in raw score. After softmax it holds more than half of all the attention.

# Softmax exaggerates the leader

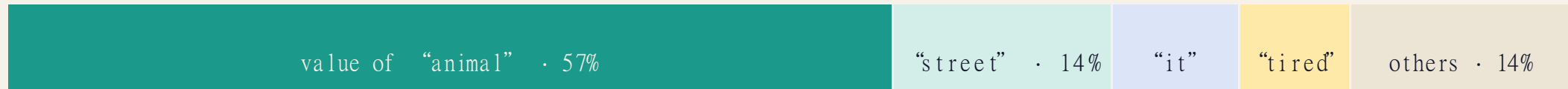


The order is the same in both charts, yet the winner's share jumps from two-thirds to almost nine-tenths.

Large scores make attention lock onto a single token and ignore the rest. Scaling in Step 2 keeps scores moderate so that attention can stay spread when it should.

## Step 4: blend the values

THE UPDATE FOR “it” IS A MIXTURE OF EVERY TOKEN'S VALUE



Each token's value is multiplied by its attention weight, and the results are added into a single new vector.

### EFFECT

#### Mostly “animal”

More than half of the mixture comes from the value of “animal”, so its qualities dominate the result.

### NOTE

#### A blend, not a pick

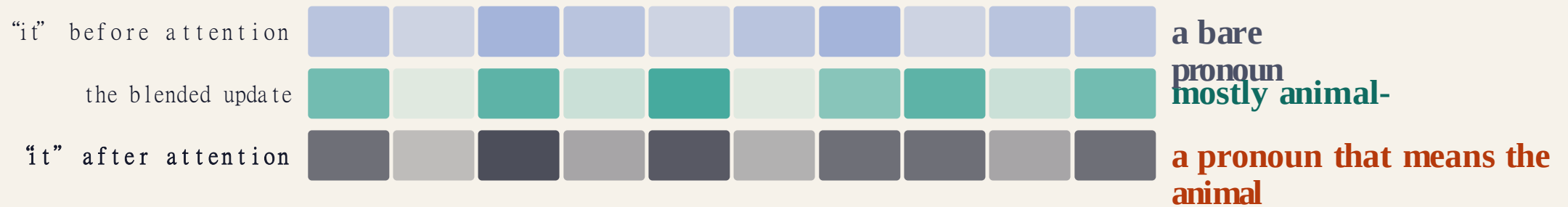
Nothing is discarded outright. Low-weight tokens still contribute a trace.

### FORM

#### Same length

The mixture is a vector of the same kind as the values that went into it.

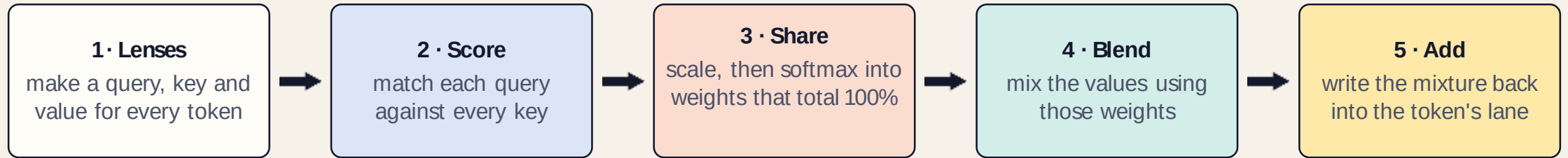
## Step 5: add the result back



The update is added to the vector “it” already had. The original information stays; the new context is layered on top.

This is the notebook idea from Part 4: attention writes a note into the token's lane rather than replacing what was there.

# Attention in five steps



That is the whole mechanism. Everything else in this part is either a restriction on it (the mask) or several copies of it (heads).

# A worked example for “it”

| Token looked at      | Match score | Attention weight | What it contributes                              |
|----------------------|-------------|------------------|--------------------------------------------------|
| animal               | 4.0         | 57.0%            | Most of the update: a living thing that can tire |
| street               | 2.6         | 14.0%            | A smaller share: a place, a noun                 |
| it                   | 2.0         | 7.7%             | A little of its own value                        |
| tired                | 1.9         | 7.0%             | A hint about state                               |
| because              | 1.0         | 2.8%             | Almost nothing                                   |
| six remaining tokens | 0.3 to 0.9  | 11.5% in total   | Traces                                           |

The weights are the exact softmax of the scores, and they add up to 100%. No step involves a rule about pronouns; the behaviour comes entirely from learned lenses.

Illustrative scores, real arithmetic.

# The attention pattern as a heatmap

ROWS: TOKEN DOING THE LOOKING · COLUMNS: TOKEN LOOKED AT

| The | cat | sat | on | the | mat |     |
|-----|-----|-----|----|-----|-----|-----|
| 40  | 30  | 10  | 5  | 10  | 5   | The |
| 25  | 35  | 20  | 5  | 5   | 10  | cat |
| 5   | 50  | 20  | 10 | 5   | 10  | sat |
| 5   | 10  | 35  | 20 | 10  | 20  | on  |
| 10  | 5   | 10  | 25 | 20  | 30  | the |
| 5   | 15  | 30  | 20 | 15  | 15  | mat |

Put every token's weights in a grid and you get the head's attention pattern.

Read along a row to see where that token looked. Every row adds up to 100.

**Row“ sat” : half its attention goes to“ cat” . A verb has found its subject.**



# Self-attention: every token at once

## EVERYONE ASKS

### Each token is a questioner

All six tokens send out a query in the same instant.

## EVERYONE ANSWERS

### Each token is also a source

The same six tokens supply the keys and values. The sentence attends to itself, hence "self"-attention.

## ONE SWEEP

### The grid is computed in one go

No token waits for another. Graphics processors fill the entire grid in parallel.

An RNN needed five steps to carry news from word one to word six. Here every token reaches every other in a single step.

# The causal mask: no peeking ahead

The cat sat on ? the mat hidden from  
“on”

## GENERATION

### While writing

When the model generates text, later tokens do not exist yet. There is nothing ahead to look at.

## TRAINING

### While learning

Training text is complete, so the next tokens are sitting on the page. Letting a token see them would be reading the answer before guessing.

A **causal mask** forces every token's attention on later tokens to zero. Each token may look only at itself and what came before.

# A masked attention pattern

THE SAME GRID WITH THE FUTURE BLOCKED

| The | cat | sat | on | the | mat |     |
|-----|-----|-----|----|-----|-----|-----|
| 100 | -   | -   | -  | -   | -   | The |
| 45  | 55  | -   | -  | -   | -   | cat |
| 8   | 62  | 30  | -  | -   | -   | sat |
| 6   | 14  | 50  | 30 | -   | -   | on  |
| 15  | 8   | 17  | 35 | 25  | -   | the |
| 5   | 15  | 30  | 20 | 15  | 15  | mat |

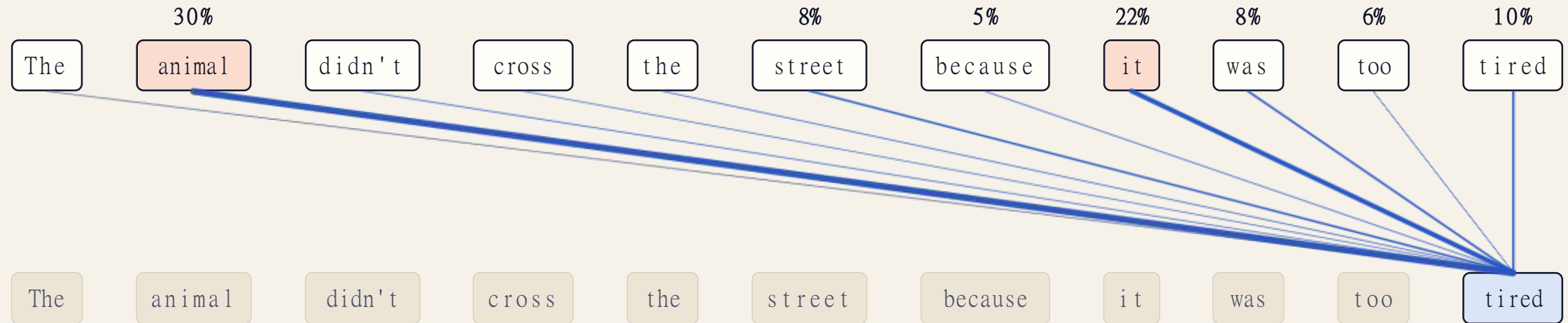
The upper-right triangle is blanked out. What remains of each row is re-shared so it still totals 100.

The first token can only attend to itself. The last token sees everything.

This triangle is the signature of every GPT-style model.

# The pronoun puzzle under a mask

TOKENS BEING LOOKED AT

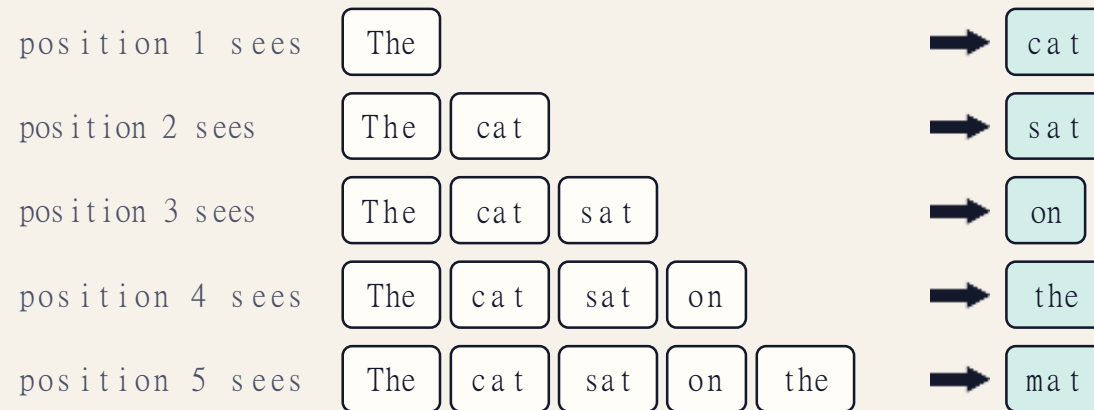


THE NEWEST TOKEN LOOKS BACK

In a GPT-style model "it" cannot see "tired", which comes later. The link is made from the other side: when "tired" arrives, it looks back at both "it" and "animal".

Information only ever flows from earlier tokens to later ones. The newest token is always the best informed.

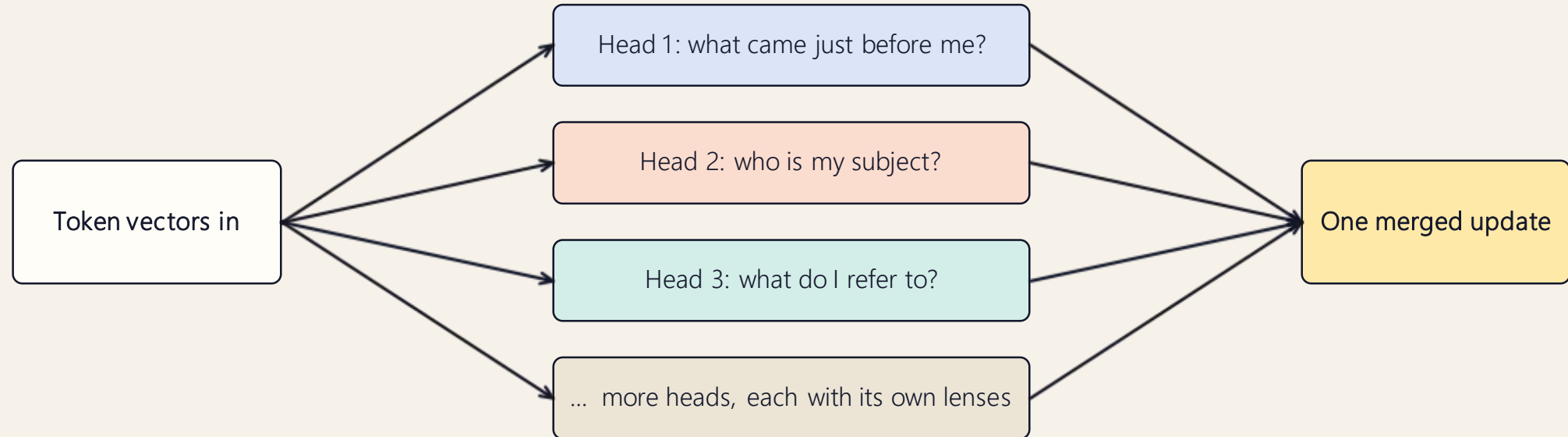
# One pass, many predictions



Because no position can see ahead, every position can be asked to predict its own next token in the same pass.

A text of a thousand tokens yields a thousand guessing exercises at once. This is a large part of why transformers train so efficiently.

# Multi-head attention: several spotlights



One set of lenses can only track one kind of relationship. So each layer runs many attention heads side by side, each free to look for something different.

GPT-2 small has 12 heads in each of its 12 layers. The original Transformer used 8 per layer.

# What heads tend to specialise in

POSITION

## Previous-token head

Each token attends to the one just before it. A building block for recognising phrases.

GRAMMAR

## Subject–verb head

Verbs attend to their subjects, even across intervening words.

MEANING

## Reference head

Pronouns attend to the nouns they stand for.

PATTERN

## Copying head

A token attends to an earlier occurrence of the same pattern, helping the model repeat names and continue lists.

Nobody assigns these jobs. They are found afterwards, when researchers inspect what trained heads are doing.

**Many heads have no neat description, and some do several jobs at once.**

# Four heads, one token

WHERE “ma t” LOOKS , HEAD BY HEAD · ILLUSTRATIVE

| The | cat | sat | on | the | ma t |        |
|-----|-----|-----|----|-----|------|--------|
|     |     |     |    |     |      | Head A |
| 1   | 1   | 2   | 4  | 88  | 4    | Head B |
| 2   | 8   | 76  | 6  | 3   | 5    | Head C |
| 3   | 78  | 8   | 2  | 3   | 6    | Head D |
| 15  | 15  | 15  | 15 | 15  | 25   |        |

- 01

**Head A**  
Looks at the previous token, “the”.
- 02

**Head B**  
Looks for the verb, “sat”.
- 03

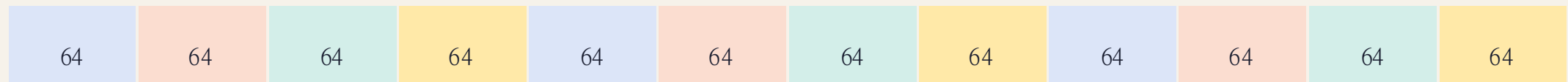
**Head C**  
Looks for the subject, “cat”.
- 04

**Head D**  
Spreads out, taking a general summary.



# Splitting the width among heads

ONE TOKEN'S 768 NUMBERS IN GPT-2 SMALL, DIVIDED AMONG 12 HEADS



Heads do not each get a full-width vector. The width is divided, so every head works with shorter queries, keys and values.

## TRADE-OFF

### Why split?

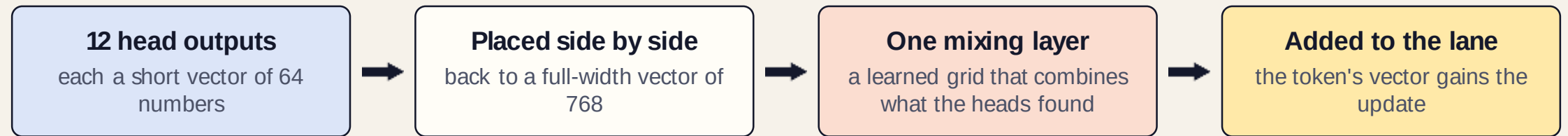
Twelve narrow heads cost about the same as one wide head, and they can follow twelve different relationships instead of one.

## GPT-2 SMALL

### The numbers

768 numbers per token, 12 heads, 64 numbers for each head.

## Merging the heads



After mixing, the layer's attention step is finished: every token's vector has absorbed what all the heads gathered for it.

# What attention does and does not do

## DOES

### It moves information

Attention carries content from one token's lane into another's. It is the only place in the model where tokens communicate.

## DOES NOT

### It does little processing

A weighted blend is a simple operation. Attention gathers the right ingredients but does not do much cooking with them.

Gathering is half the job. Once a token has collected context, something needs to think about it. That is the second half of every block, coming up in Part 7.

# The cost of comparing every pair

**100**

comparisons per head  
for 10 tokens

**10,000**

comparisons per head  
for 100 tokens

**1,000,000**

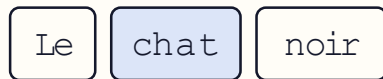
comparisons per head  
for 1,000 tokens

Every token is compared with every token. Ten times more text means a hundred times more comparisons.

This steep growth is the main reason context windows have limits.

# Cross-attention

## SELF-ATTENTION



looks at its own sequence

## CROSS-ATTENTION



looks at a different sequence

The mechanism is identical. The only change is where keys and values come from: another piece of text, such as the sentence being translated.

The original Transformer used it to connect its decoder to its encoder. Most chat models today need only masked self-attention; Part 10 explains why.

# Common misconceptions about attention

| A tempting belief                          | A more accurate picture                                                     |
|--------------------------------------------|-----------------------------------------------------------------------------|
| It works like human attention              | It is a weighted average with learned weights; the name is a loose metaphor |
| High weight means “this caused the answer” | Weights show where information moved, not everything the model did with it  |
| Each head has one clear job                | Some do; many mix several roles or resist description                       |
| Attention is the whole model               | More parameters sit in the feed-forward layers than in attention            |
| The model reads left to right              | All positions are processed together; the mask only limits who can see whom |

# Part 6 in six ideas

01

## The job

Let each token gather the context it needs from other tokens.

02

## Query, key, value

What I seek, what I advertise, what I hand over: three lenses on each vector.

03

## Five steps

Lenses, score, share with softmax, blend the values, add back.

04

## The mask

Tokens may look only at themselves and earlier tokens.

05

## Many heads

Several attention patterns run in parallel and are merged.

06

## The price

Every pair is compared, so cost climbs steeply with length.

# Inside a block

Attention is half of every block. The other half is a small network that thinks about what attention gathered, plus two devices that keep a tall stack stable.

## IN THIS PART

Communicate, then compute

The feed-forward network

Residual connections

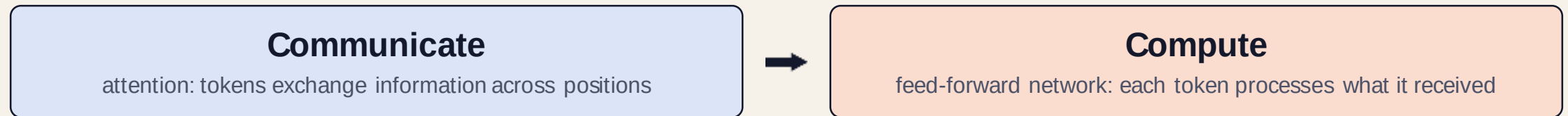
Layer normalisation

One full block

Depth and size



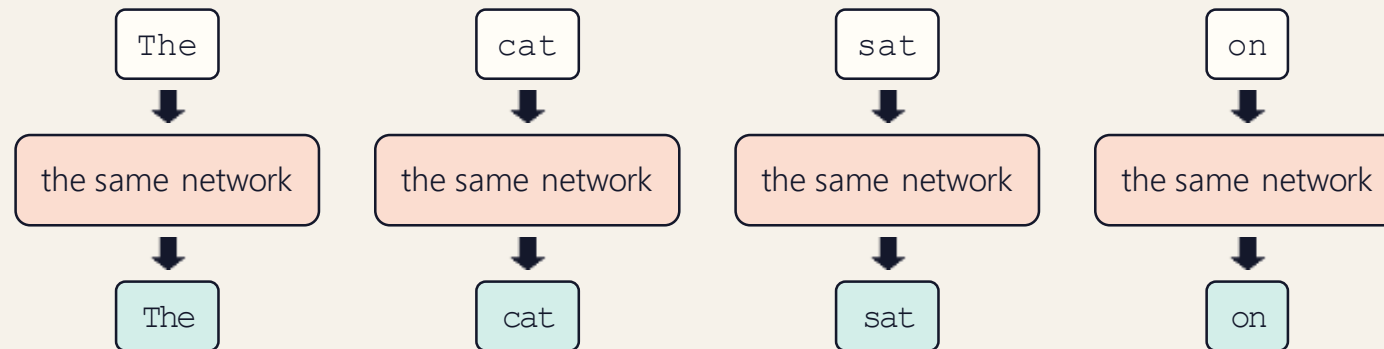
# The two-step rhythm of a block



Every block does these two things in this order, and the whole model is the same block repeated many times.

**A meeting, then desk work. Then another meeting, then more desk work.**

# The feed-forward network: private thinking

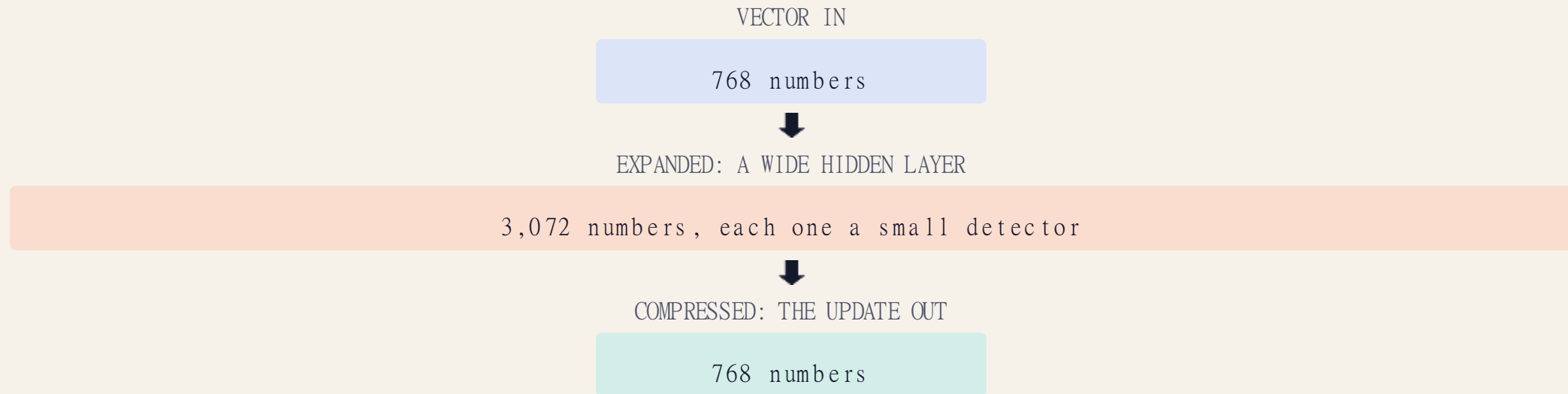


Each token's vector goes through the feed-forward network on its own.

There is no talking between positions in this step. The same network, with the same dials, is applied to every token separately.

**Attention decides what to gather.  
network decides what to make of it.**

# Expand, switch, compress



The network spreads the vector across a layer four times wider, applies a switch to each number, then squeezes the result back to the original width.

Sizes shown are GPT-2 small's. The four-to-one ratio comes from the original paper.

# The activation: an “only if” switch

|                   |     |      |     |      |     |      |     |      |     |
|-------------------|-----|------|-----|------|-----|------|-----|------|-----|
| before the switch | 0.8 | -0.5 | 1.2 | -1.1 | 0.3 | -0.2 | 0.9 | -0.7 | ... |
| after the switch  | 0.8 | 0.0  | 1.2 | 0.0  | 0.3 | 0.0  | 0.9 | 0.0  | ... |

Each number in the wide layer passes through a simple gate: positive signals go through, negative ones are shut off.

## PURPOSE

### Why it matters

Without this gate, a stack of layers would collapse into one plain layer. The switch is what lets depth add real power.

## NAMES

### In practice

The on/off version is called ReLU. GPT models use GELU, a smoother curve with the same spirit.

# Hidden units as pattern detectors

UNIT 214

Fires for capital cities

Stays silent for most tokens. Lights up when the vector says “this is about a national capital”.

UNIT 1,507

Fires inside quotations

Switches on between an opening quote mark and its closing partner.

UNIT 2,930

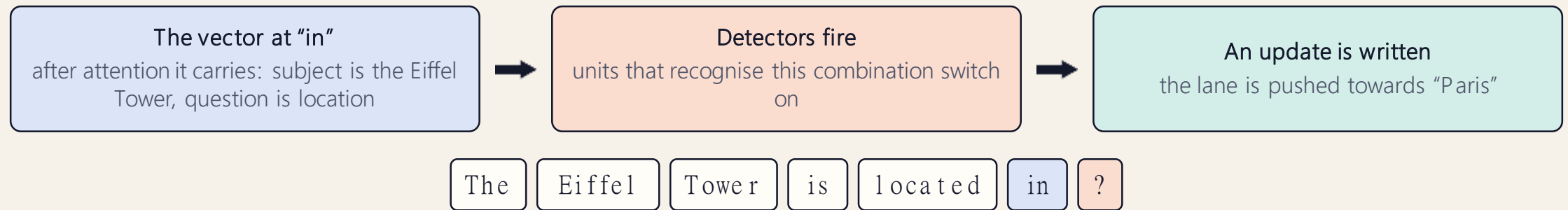
Fires for past-tense context

Responds when the surrounding sentence is describing something already finished.

Each of the thousands of hidden units watches for one pattern in the token's vector. When its pattern appears, the unit switches on and contributes its own small update.

Invented examples, to give the flavour. Real units are often messier.

# Feed-forward layers as a memory for facts



Attention assembled the question. The feed-forward layer supplies the answer from what was stored in its dials during training.

Research has traced the recall of many simple facts to feed-forward layers in the middle of the stack.

# Where a block keeps its parameters

THE DIALS OF ONE GPT-2 SMALL BLOCK

Attention · about one-third

Feed-forward network · about two-thirds

Attention gets the fame, but most of a block's learned numbers sit in its feed-forward network.

## COMMUNICATION

### Attention

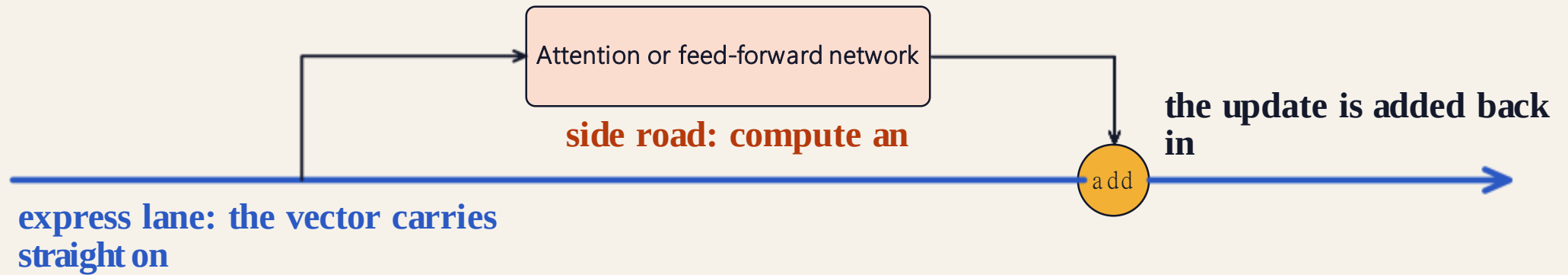
Around 2.4 million dials per block: the query, key and value lenses and the mixing layer.

## COMPUTATION AND MEMORY

### Feed-forward network

Around 4.7 million dials per block: the expanding grid and the compressing grid.

# Residual connections: the express lane



No layer is allowed to overwrite a token's vector. The layer computes on a copy, and its output is added to the original.

This bypass is called a **residual** or **skip connection**. Every attention step and every feed-forward step in the model is wrapped in one.



# Why the express lane matters

## NOTHING IS LOST

### **Layers can only add**

If a layer has nothing useful to say, it can add almost nothing, and the vector passes through intact.

## LEARNING REACHES THE BOTTOM

### **Blame travels the same road**

During training, correction signals flow back along the express lane, reaching early layers without fading.

## LAYERS COOPERATE

### **Refine, do not rebuild**

Each layer sees everything written so far and contributes one more small improvement.

The idea came from image recognition in 2015, where it first made networks with over a hundred layers trainable. Transformers depend on it.

# Layer normalisation: keeping numbers in range

|                           |      |      |      |       |      |     |
|---------------------------|------|------|------|-------|------|-----|
| a vector that has drifted | 12.0 | -7.5 | 30.2 | -18.4 | 3.1  | 9.9 |
| after normalisation       | 0.5  | -0.8 | 1.6  | -1.5  | -0.1 | 0.3 |

After many additions the numbers in a lane can grow large or lopsided. Normalisation re-centres them around zero and resets their spread to a standard size.

## WHAT IT KEEPS

### The pattern survives

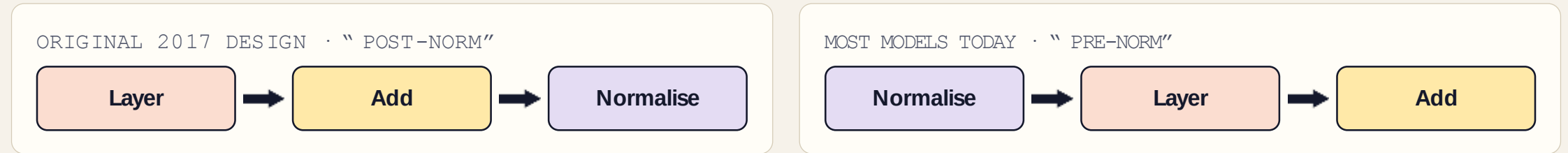
The biggest number is still the biggest and the signs barely change. Only the overall volume is reset.

## WHY

### Why it helps

Every layer receives inputs in a familiar range, which keeps training steady in very deep stacks.

# Where the normaliser sits



The original paper normalised after each addition. Later work found it better to normalise the copy going into each layer and leave the express lane itself untouched.

Pre-norm makes very deep models much easier to train. GPT-2 and its successors use it.

# Dropout: practising with random blackouts



During training, a random handful of signals is switched off at every step, a different handful each time.

## PURPOSE

### Why

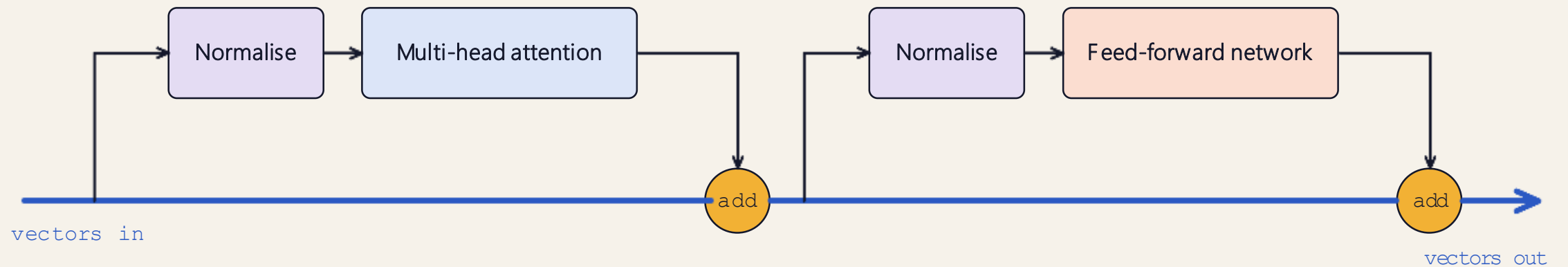
The model cannot lean on any single pathway, so it spreads knowledge around and copes better with text it has not seen.

## TIMING

### When

Only while training. When the model is in use, nothing is switched off.

# One complete block



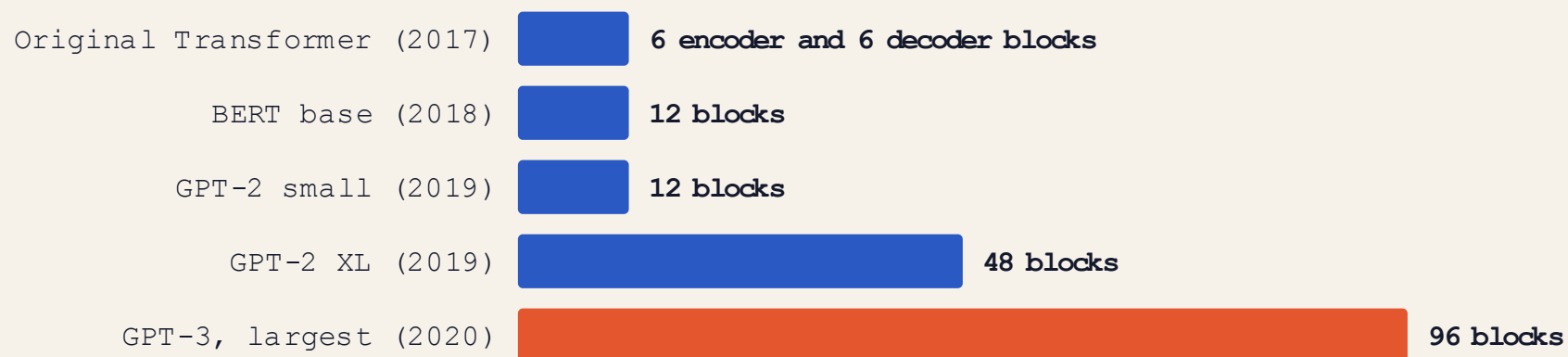
Two side roads off one express lane. First attention gathers context and adds it in; then the feed-forward network processes the result and adds its own update.

This drawing shows the pre-norm layout. Each of the five tokens in our sentence has its own lane running through this same block.

# What each part contributes

| Part                 | Its job                                               | Everyday picture                              |
|----------------------|-------------------------------------------------------|-----------------------------------------------|
| Normalise            | Reset the volume of the numbers before each layer     | Tuning an instrument before playing           |
| Multi-head attention | Let each token collect context from earlier tokens    | A meeting where everyone consults their notes |
| Add (first)          | Write what was gathered into the token's lane         | Adding to the minutes                         |
| Feed-forward network | Process the gathered context; recall stored knowledge | Desk work after the meeting                   |
| Add (second)         | Write the conclusions into the lane                   | Filing the result                             |

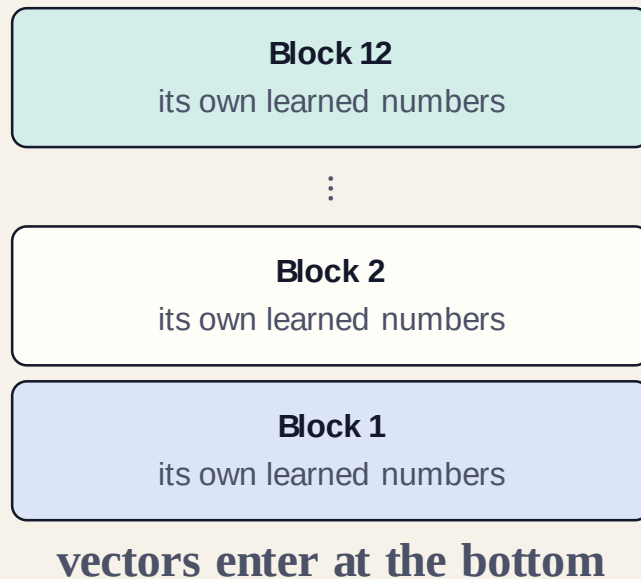
# Stacking blocks: depth



The output of one block is the input of the next. The number of blocks is the model's **depth**.

**More depth means more rounds of gather-then-think before the model must answer**

# Same shape, different dials



Every block has an identical layout: attention, feed-forward network, two additions.

What differs is the settings. Each block has its own lenses and its own detectors, learned separately, so each can specialise in a different kind of work.

**One blueprint, built twelve times, furnished twelve different ways.**



# What tends to happen at each depth

## EARLY BLOCKS

### Surface and structure

Word forms, neighbouring tokens, basic grammar. Pieces of a split word are joined back together.

## MIDDLE BLOCKS

### Meaning and facts

Who did what to whom, what a pronoun refers to, facts about the things mentioned.

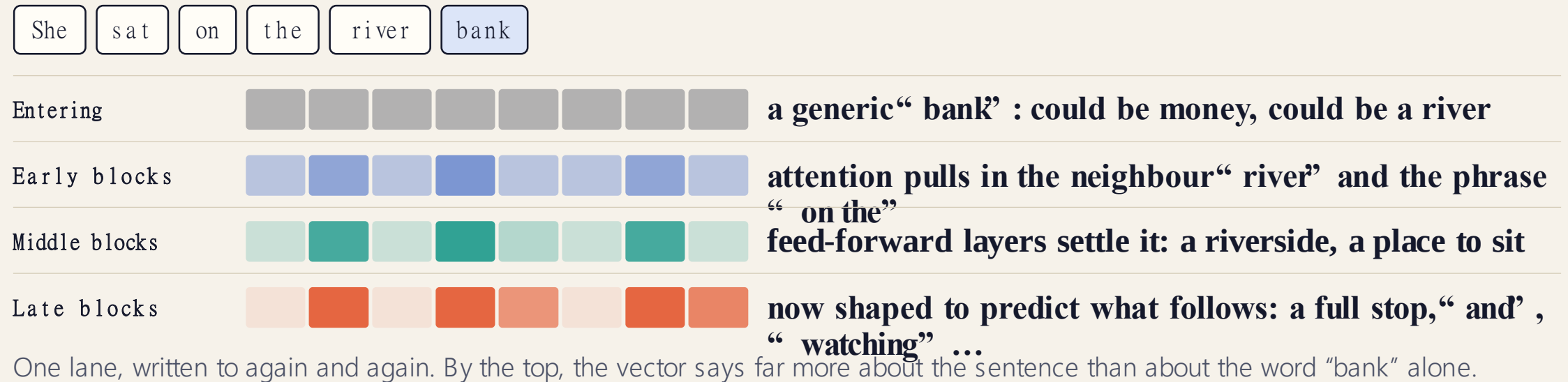
## LATE BLOCKS

### Preparing the answer

The lane is steered towards the specific token that should come next, in the right form and style.

This is a broad tendency reported by researchers who probe trained models, not a hard division. Many abilities are spread across depths.

# Following “bank” up the stack



# Three size knobs

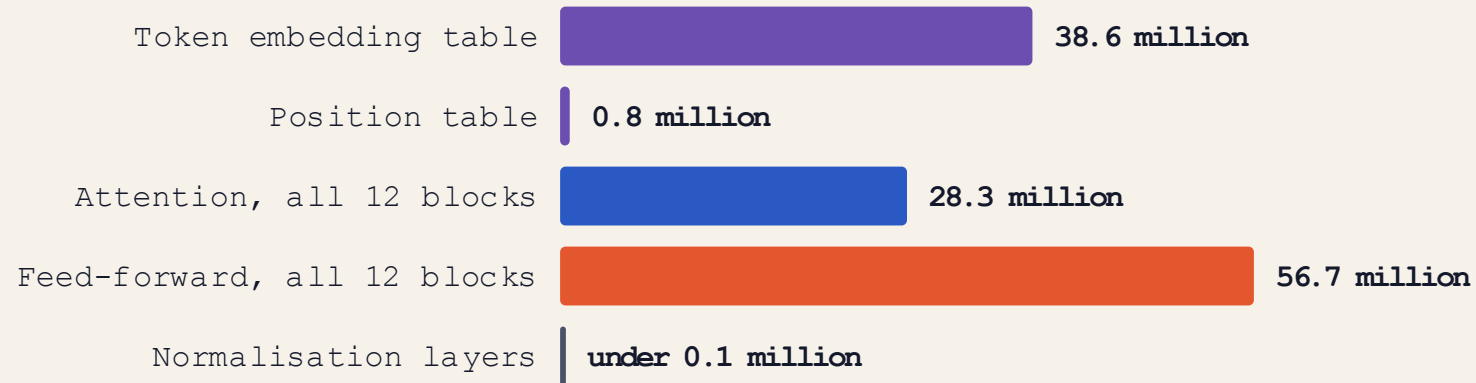
| Model                             | Blocks (depth) | Numbers per token (width) | Heads per block | Parameters        |
|-----------------------------------|----------------|---------------------------|-----------------|-------------------|
| Original Transformer, base (2017) | 6 and 6        | 512                       | 8               | about 65 million  |
| BERT base (2018)                  | 12             | 768                       | 12              | about 110 million |
| GPT-2 small (2019)                | 12             | 768                       | 12              | about 124 million |
| GPT-2 XL (2019)                   | 48             | 1,600                     | 25              | about 1.5 billion |
| GPT-3, largest (2020)             | 96             | 12,288                    | 96              | about 175 billion |

Depth is the number of rounds, width is how much each lane can hold, and heads are how many relationships each round can track.

Same blueprint at every size.

# Counting GPT-2 small's parameters

MILLIONS OF LEARNED NUMBERS, BY COMPONENT



## 124 million

parameters in total

Every one of them began as a random number and was set by training. None was written by a person.

## Part 7 in six ideas

01

### Two steps per block

Attention communicates; the feed-forward network computes.

02

### Expand, switch, compress

A wide layer of detectors, gated, then squeezed back.

03

### Memory lives here

Most parameters, and much factual recall, sit in feed-forward layers.

04

### Residual connections

Layers add to a token's lane and never overwrite it.

05

### Normalisation

Resets the scale of the numbers so deep stacks stay stable.

06

### Depth, width, heads

The same block repeated; three knobs set the model's size.

# From vectors to words

After the last block, each token's lane holds a refined vector. This part turns that vector into a probability chart and then into one chosen token.

## IN THIS PART

The final vector

Scores for every token

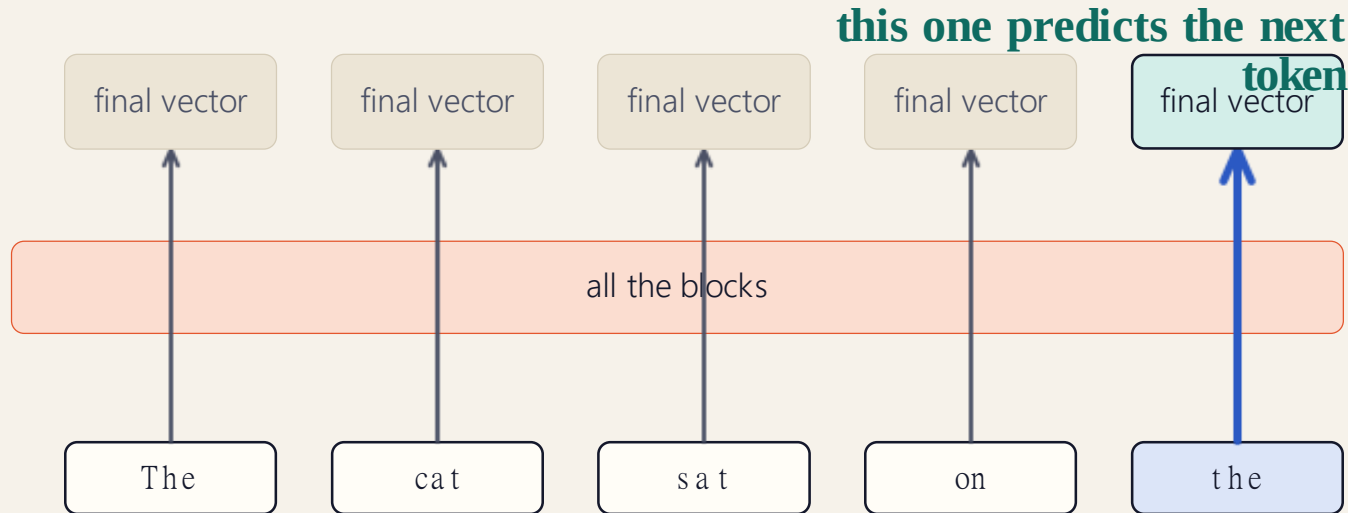
Softmax again

Temperature

Top-k and top-p

The generation loop

# The last lane holds the prediction



At the top of the stack, the vector above each token is the model's summary of "what should come after me?"

To continue the text, only the final token's vector is needed. It has had access, through attention, to everything before it.

# Unembedding: comparing against the vocabulary

THE FINAL VECTOR



held up against every token's



|                                             |  |  |  |  |  |  |                     |
|---------------------------------------------|--|--|--|--|--|--|---------------------|
| mat                                         |  |  |  |  |  |  | <b>strong match</b> |
| floor                                       |  |  |  |  |  |  | good match          |
| sofa                                        |  |  |  |  |  |  | fair match          |
| banana                                      |  |  |  |  |  |  | poor match          |
| because                                     |  |  |  |  |  |  | poor match          |
| ... and every other token in the vocabulary |  |  |  |  |  |  |                     |

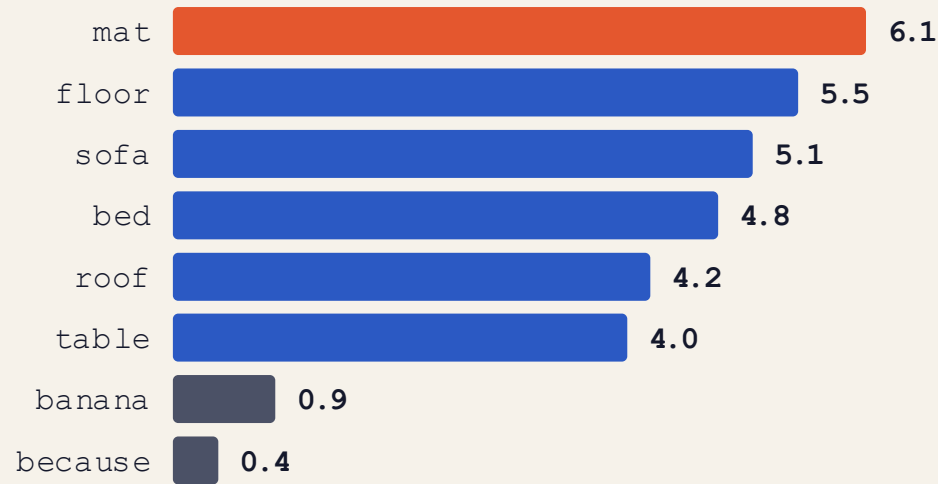
The model runs the similarity test one last time: how well does the final vector line up with each token in the vocabulary?

This is the embedding lookup in reverse. At the input, a token became a vector. Here, a vector is matched back to tokens.



# Logits: one raw score per token

SCORES FOR THE TOKEN AFTER "The cat sat on the" · ILLUSTRATIVE

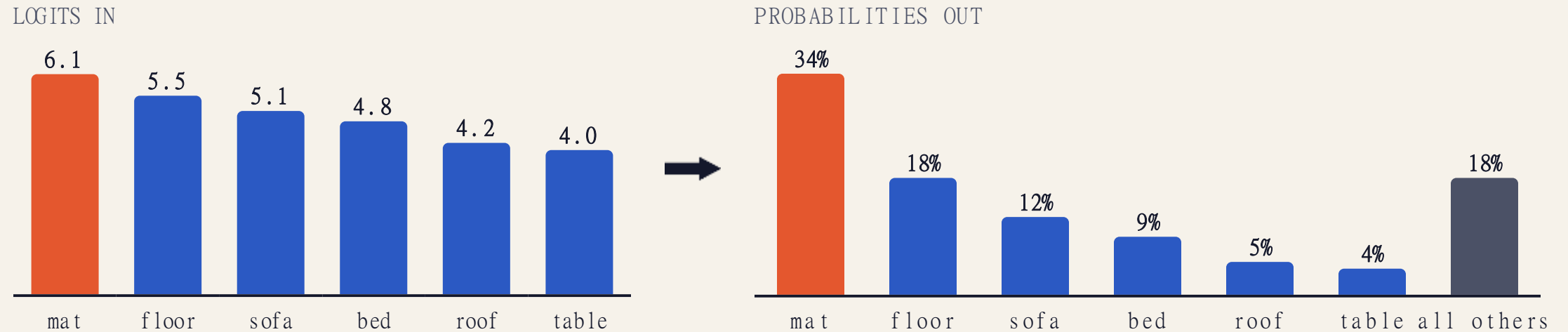


The result is a long list of raw scores, one for every token the model knows.

These scores are called **logits**. They can be any size, positive or negative, and they do not yet add up to anything meaningful.

For GPT-2 this list has 50,257 entries, produced at every step.

# Softmax, a second time



The same recipe that produced attention weights now turns logits into probabilities: all positive, adding up to 100%.

This is the chart from Part 1. We have now seen every step that produces it.

# Weight tying: one table, two jobs

AT THE INPUT

## Token to vector

Look up the token's row to get its starting vector.

←  
the same table  
→

AT THE OUTPUT

## Vector to scores

Compare the final vector with every row to score every token.

Many models, GPT-2 among them, use a single table for both directions. A token's meaning on the way in is also what the model aims at on the way out.

**Sharing the table saves tens of millions of parameters.**

# From a chart to one token

The network's work ends with the probability chart. Choosing a single token from it is a separate step, carried out by ordinary software.

## SIMPLEST

### Greedy

Always take the most probable token.

## VARIED

### Sampling

Draw a token at random, in proportion to its probability.

## FILTERED

### Top-k and top-p

Sample, but only from the plausible candidates.

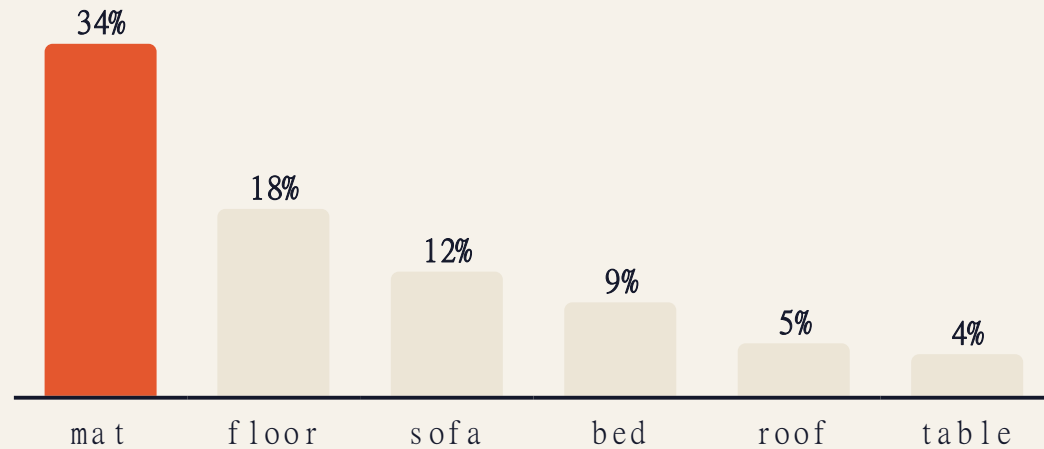
## A DIAL

### Temperature

Reshape the chart before drawing from it.

These settings change the style of the output without touching a single dial inside the model.

# Greedy decoding: always the favourite



Pick the tallest bar, every time.

## STRENGTH

### Predictable

The same prompt always gives the same continuation.

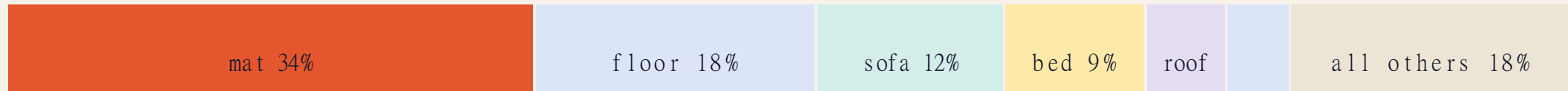
## WEAKNESS

### Flat and repetitive

Text tends to be bland, and can fall into loops, repeating a phrase again and again.

# Sampling: rolling weighted dice

THE CHART LAID OUT AS A STRIP · EACH TOKEN'S SLICE MATCHES ITS PROBABILITY



Throw a dart at the strip and take the token it lands on. “mat” wins about a third of the time; the others get their turn.

## STRENGTH

### More natural

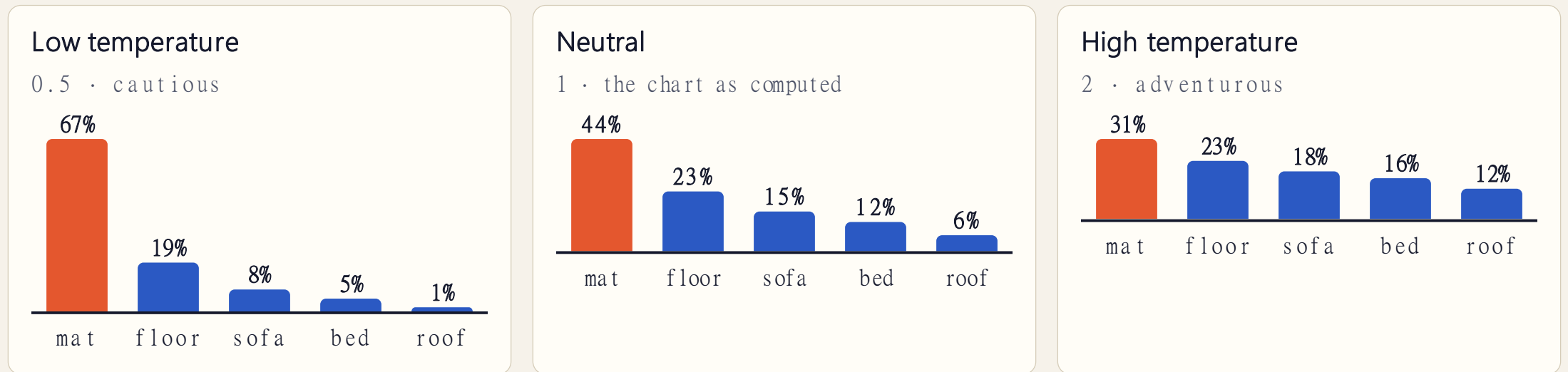
Human writing does not always take the single most likely word. Sampling restores that variety.

## WEAKNESS

### Occasional strays

One time in many, the dart lands on a very unlikely token and the text veers off.

# Temperature: sharpening or flattening

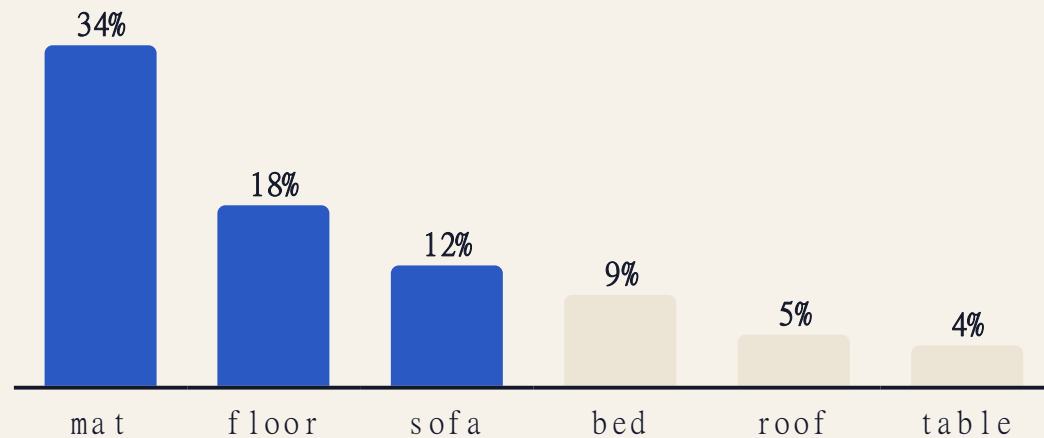


Temperature rescales the logits before softmax. Low values exaggerate the leader; high values level the field.

At the lowest setting this becomes greedy decoding. Illustrative chart, restricted to the top five tokens.

# Top-k: keep the k best

TOP-K WITH K SET TO 3



Keep the k most probable tokens, discard the rest, and sample among the survivors.

This removes the long tail of unlikely tokens that cause sampling to go astray.

**The catch: k is fixed. Sometimes three options are too many, sometimes far too few.**



# Top-p: keep enough to cover p

TOP-P WITH P SET TO 70% · ADD TOKENS FROM THE LEFT UNTIL THE RUNNING TOTAL PASSES 70%



↑ cut here: four tokens

Instead of a fixed count, keep the smallest group of top tokens whose probabilities together reach a chosen share, then sample within it.

## STRENGTH

It adapts

When the model is confident, one or two tokens are enough. When it is unsure, many stay in play.

## NAMES

Also called

Nucleus sampling. It is the most widely used filter, often combined with temperature.

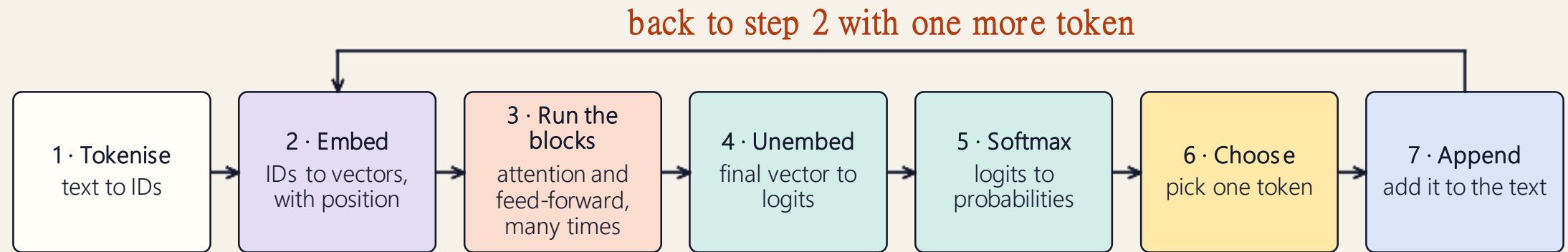
# Beam search: several drafts at once



Follow a few candidate continuations side by side and keep whichever are most probable overall.

Useful when there is one right answer to find, as in translation. Chat models rarely use it, because it favours safe, generic text.

# The generation loop, step by step



One trip round this loop produces one token. A reply of five hundred tokens is five hundred trips.

Step 1 happens once, for the prompt. After that, the loop runs from step 2 to step 7 until something tells it to stop.

# Knowing when to stop

## THE MODEL DECIDES

### End-of-text token

When the model judges the reply complete, the special end token becomes the most probable choice. Once it is picked, the loop ends.

## A LIMIT IS REACHED

### Maximum length

The software sets a cap on new tokens. If the cap is hit first, the reply is cut off, sometimes mid-sentence.

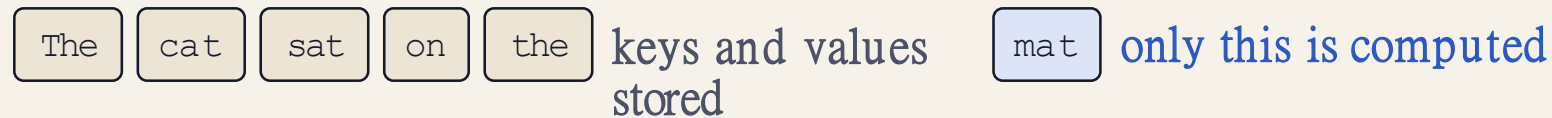
## A SIGNAL APPEARS

### Stop sequences

A developer can name certain strings. If the output produces one, generation halts at that point.

Left alone, the loop would run forever. Something outside the network has to end it.

# The KV cache: not redoing old work



Because of the causal mask, an earlier token never sees later ones. Its keys and values stay the same no matter what is appended.

## THE TRICK

### Store once

Keep every earlier token's keys and values from the previous trip round the loop.

## THE SAVING

### Compute one lane

On each new trip, only the newest token needs its vectors computed. It attends to the stored ones.

## THE COST

### Pay in memory

The cache grows with the conversation, which is one reason long contexts are expensive to serve.

# Why answers vary, and why they can be wrong

## VARIATION

### Chance is part of the design

With sampling switched on, the same prompt can lead down different paths. One different token early on changes everything after it.

## FLUENT ERRORS

### Plausible is not the same as true

The loop always yields a likely-sounding next token. Nothing in it checks facts, and there is no built-in way to stay silent when unsure.

Both behaviours follow directly from the mechanism. Part 11 returns to them.

# Part 8 in six ideas

01

## The final vector

The last token's lane, after all blocks, encodes what should come next.

02

## Logits

That vector is compared with every vocabulary entry, giving one score each.

03

## Softmax

Scores become probabilities that add up to 100%.

04

## Decoding

Software picks a token: greedily, or by sampling with temperature, top-k or top-p.

05

## The loop

Append the token and run again, until a stop condition.

06

## The cache

Earlier tokens' keys and values are stored, so each step computes one new lane.

# Training

So far the dials were assumed to be well set. This part explains how they get that way: by guessing the next token an enormous number of times and correcting after each guess.

## IN THIS PART

Inputs and targets

The loss

Backpropagation

Gradient descent

Data and compute

Fine-tuning



# The training objective

Take real text. At every position, ask the model for the next token. Reward it for putting high probability on the token that was actually there.

## ONE RULE

### Nothing else is taught

No grammar lessons, no fact sheets, no labelled examples. The only signal is how well the next token was predicted.

## FREE ANSWERS

### Text marks itself

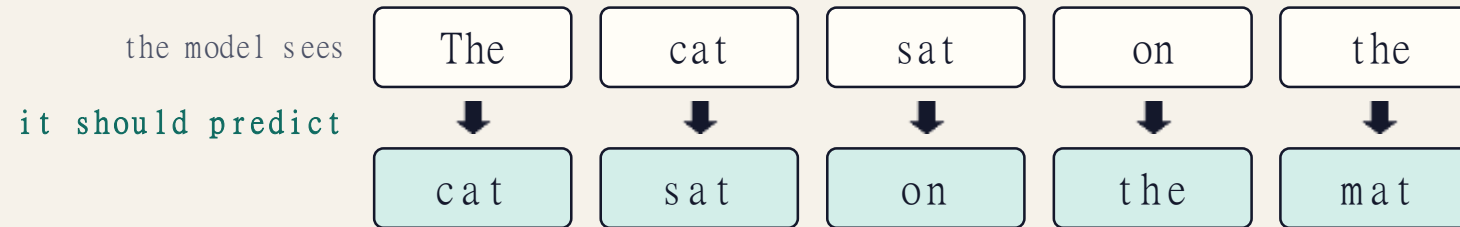
Because the next token is already on the page, any document can be used as practice material without human effort.

## EVERY POSITION

### All at once

Thanks to the causal mask, one pass over a passage produces a prediction, and a correction, for every token in it.

# Inputs and targets: one text, shifted by one



The target row is the input row moved one place to the left. Under “The” the right answer is “cat”; under “cat” it is “sat”; and so on.

**Six tokens of text give five graded guesses in a single pass.**

# The loss: a penalty for surprise

PROBABILITY THE MODEL GAVE TO THE CORRECT TOKEN, AND THE PENALTY



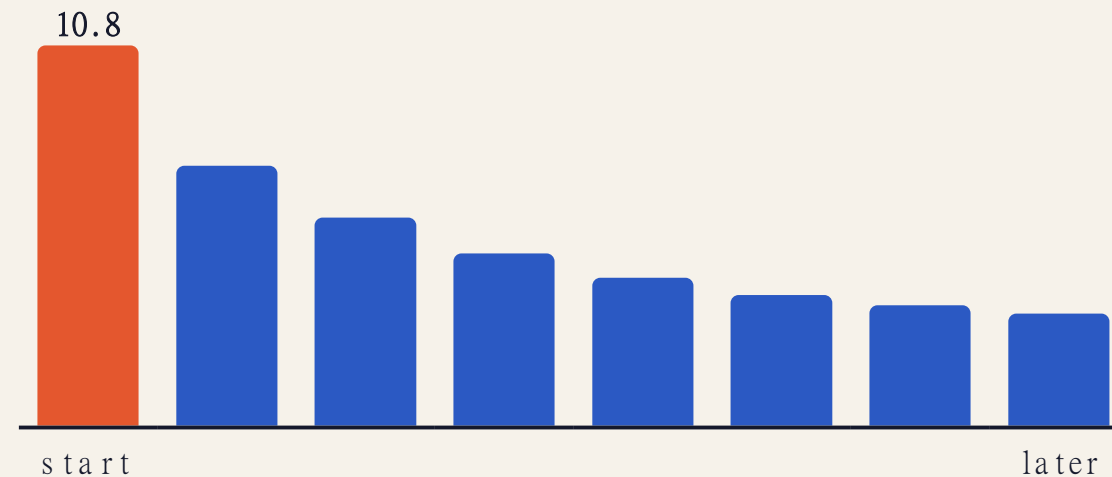
The penalty depends only on the probability given to the token that really came next.

High probability, small penalty. Low probability, large penalty, and it grows sharply as the probability approaches zero.

**Its formal name is cross-entropy loss.  
Lower is better.**

# Loss over the course of training

AVERAGE PENALTY PER TOKEN · SHAPE IS ILLUSTRATIVE



A fresh model guesses blindly. With 50,257 tokens to choose from, blind guessing earns a penalty of about 10.8.

## 01 Fast early gains

Common words and spacing are learned almost at once.

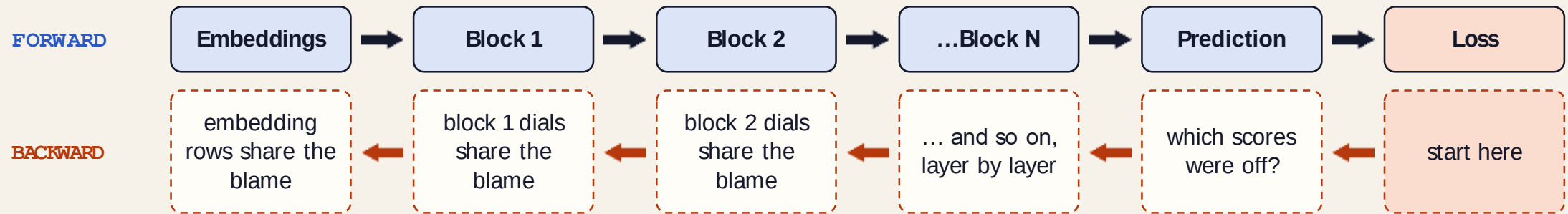
## 02 Slower middle

Grammar and frequent phrases follow.

## 03 Long tail

Facts, style and reasoning patterns keep improving for a very long time.

# Backpropagation: tracing the blame

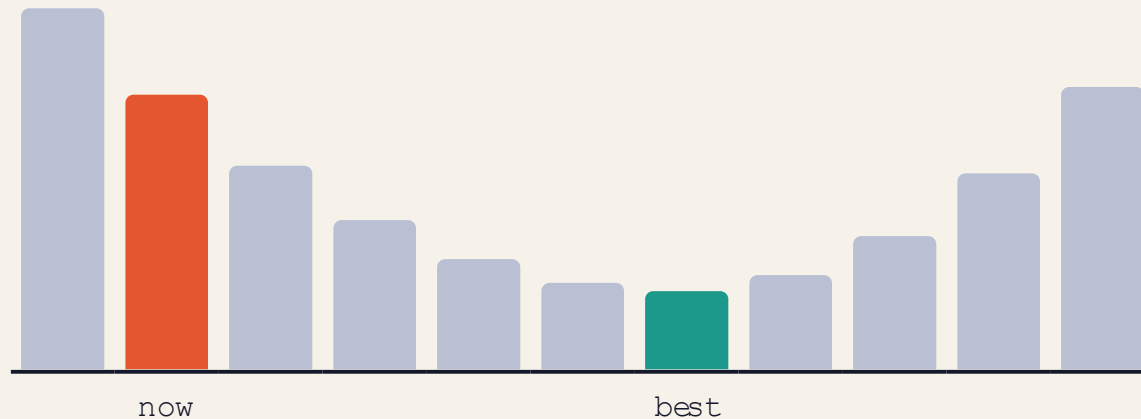


After the forward pass measures the error, a backward pass works out, for every single dial, which way to turn it to make that error smaller.

It starts at the loss and moves back layer by layer. The residual express lane lets this signal reach the earliest layers at full strength.

# Gradient descent: downhill in the fog

ERROR FOR DIFFERENT SETTINGS OF ONE DIAL



Imagine standing on a hillside in thick fog, wanting to reach the valley floor.

You cannot see the valley, but you can feel which way the ground slopes under your feet. Take a small step downhill, feel again, and repeat.

Backpropagation supplies the slope. Gradient descent takes the step.

# The learning rate: stride length

TOO SMALL

## Shuffling

Each step barely moves the dials. Training crawls and may never get anywhere useful in the time available.

ABOUT RIGHT

## Steady walking

Steps are large enough to make progress and small enough to stay on the slope.

TOO LARGE

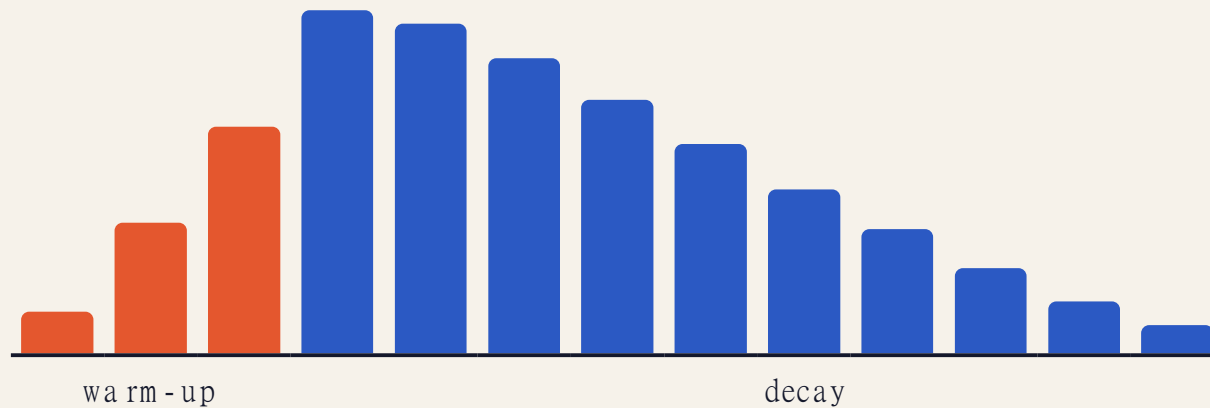
## Leaping

Each step overshoots the valley and lands on the far side. The error bounces around or blows up.

How far to move the dials at each step is a setting chosen by the people running the training. It is one of the most sensitive choices they make.

# Warm-up, then decay

STRIDE LENGTH FROM THE FIRST TRAINING STEP TO THE LAST · ILLUSTRATIVE

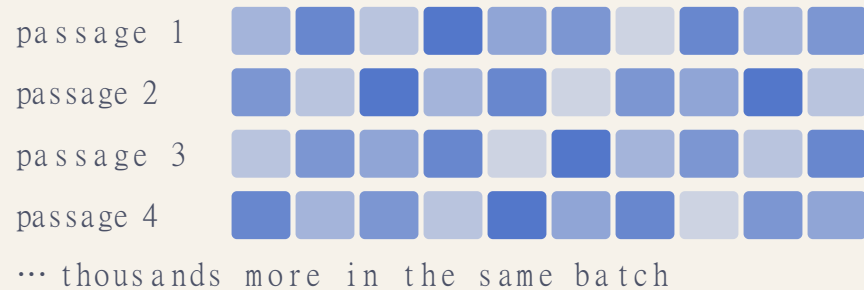


The stride is not kept constant.

- 01 Warm-up**  
Begin with tiny steps while the dials are still random and the slopes unreliable.
- 02 Peak**  
Take the largest steps once the model has found its footing.
- 03 Decay**  
Shrink the steps gradually to settle into a good spot.



## Batches: many examples per step



The dials are not adjusted after every single passage. Many passages are processed together, and their blame is averaged.

One passage alone gives a noisy, lopsided view of the slope. The average of many is a much steadier guide, and processors handle the whole batch in parallel.

# Adam: a walker with memory

## MOMENTUM

### Remember recent directions

Keep a running average of recent slopes and move along that, so one odd step does not knock the walk off course.

## PER-DIAL STRIDE

### Adapt to each dial

Dials that have been getting large, erratic signals take shorter steps; quiet ones take longer steps.

## WEIGHT DECAY

### A gentle pull to zero

Every dial is nudged slightly towards zero so that none grows extreme. With this added, the method is called AdamW.

Plain gradient descent is rarely used as is. Almost all transformers are trained with Adam or AdamW, which add these refinements.

# Teacher forcing: the true history

## TRUE HISTORY

### During training

The cat sat ?

The model always sees the true earlier tokens, whatever it would have guessed itself.

## OWN HISTORY

### During use

The cat ran ?

The model sees its own earlier choices. If one was odd, it must carry on from there.

Training on the true history is what lets every position be graded in one pass. The price is a gap between practice and performance: in use, small slips can compound.

# The training text

WEB

## Public web pages

Articles, forums, reference sites.

BOOKS

## Long-form writing

Fiction and non-fiction, for sustained structure.

CODE

## Source code

Programs and their documentation.

DIALOGUE

## Conversations

Questions and answers, discussions, transcripts.

Pretraining corpora for large models run from hundreds of billions to trillions of tokens, far more than a person could read in many lifetimes.

The text is cleaned first: duplicates removed, low-quality pages filtered, formats made consistent. What goes in shapes everything the model can do.

# Compute: why graphics processors

## THE WORK

### Grids of numbers

Every step in a transformer is arithmetic on large tables of numbers.

## THE TOOL

### Thousands of cores

A graphics processor performs the same simple operation on many numbers at the same moment.

## THE MATCH

### A natural fit

All tokens and all passages in a batch are processed together, so almost nothing waits in a queue.

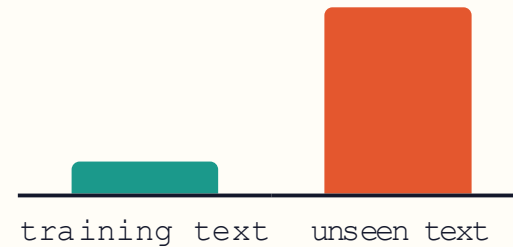
Training a large model occupies thousands of these processors for weeks or months. Removing recurrence in 2017 is what made such scale practical.

# Memorising versus generalising

Healthy: both penalties low



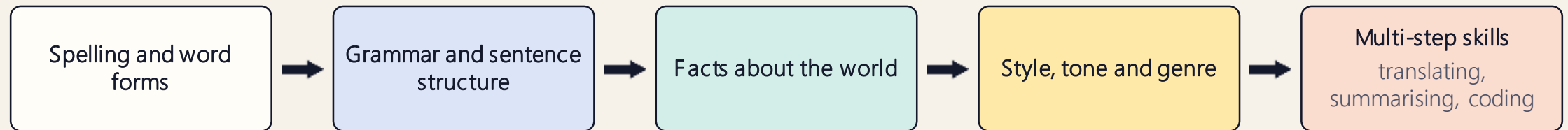
Memorising: a widening gap



The aim is to do well on text the model has never seen.

Some text is held back and never used for training. If the penalty keeps falling on training text while rising on held-back text, the model is reciting, not learning.

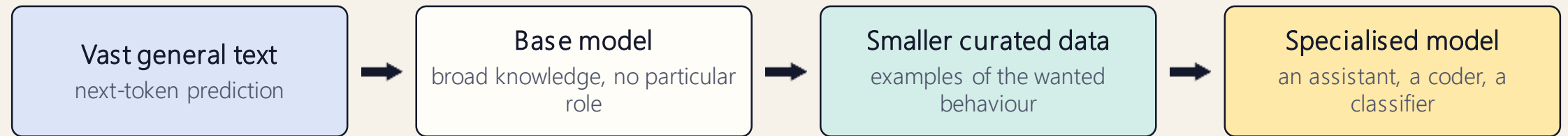
# What emerges from next-token practice



None of these was a training goal. Each appears because it makes next-token prediction more accurate.

**Roughly in this order, though the stages overlap heavily.**

# Pretraining, then fine-tuning



Pretraining is long, costly and done once. Fine-tuning continues the same guess-and-correct process on a narrower set of examples, adjusting dials that are already well set.



# Teaching a model to be an assistant

## BEFORE

A base model continues

What is the capital of France?

What is the capital of Spain? What is the capital of Italy? ...

It treats the question as the start of a quiz sheet and writes more of the sheet.

## AFTER

A tuned model answers

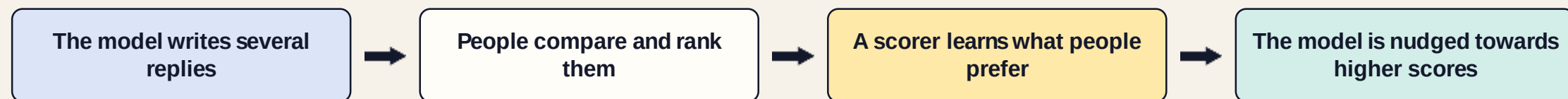
What is the capital of France?

The capital of France is Paris.

It has learned, from many example dialogues, that a user turn is followed by a helpful reply.

The examples wrap each turn in role tokens. After enough of them, “continue the text” and “answer the user” become the same thing.

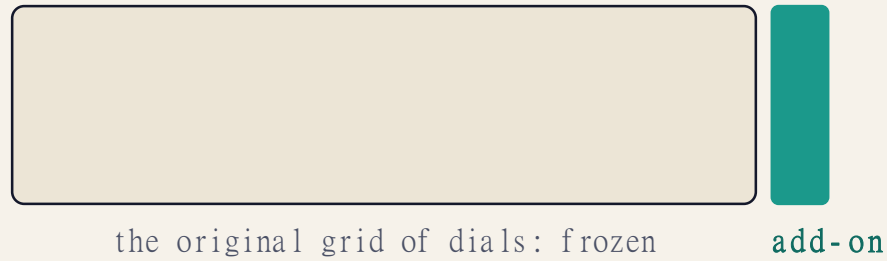
# Learning from human preferences



Next-token prediction teaches what text is likely. Feedback from people teaches which likely text is actually wanted: clear, accurate, appropriately careful.

One common form of this is called reinforcement learning from human feedback.

# Small add-ons instead of full retraining



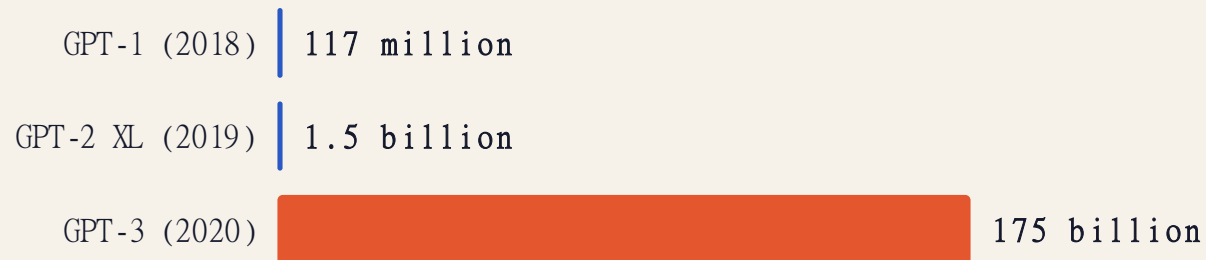
Adjusting billions of dials for every new task is expensive.

A popular shortcut, LoRA (2021), freezes the whole model and trains a pair of thin extra grids beside each large one. Only the add-ons learn.

The add-on can be well under one percent of the model's size.

# Scale: the dependable lever

PARAMETERS, DRAWN TO THE SAME SCALE



Across this period the recipe barely changed. The models were made larger, trained on more text, with more compute.

Researchers found that the loss falls smoothly and predictably as these three are increased together, which made investing in ever-larger models a calculated bet.

# Part 9 in six ideas

01

## One objective

Predict the next token at every position of real text.

02

## The loss

A penalty that is large when the true token was given low probability.

03

## Backpropagation

Works out, for every dial, which way to turn it.

04

## Gradient descent

Take a small step downhill; repeat. Adam and schedules refine the stepping.

05

## Scale

Vast text, parallel hardware and more parameters reliably lower the loss.

06

## Fine-tuning

Further training on curated examples turns a base model into an assistant.

# The model family

The 2017 design had two halves. Later models kept one half or the other, and the half that writes turned out to be the foundation of modern LLMs.

## IN THIS PART

Encoder and decoder

Readers: BERT

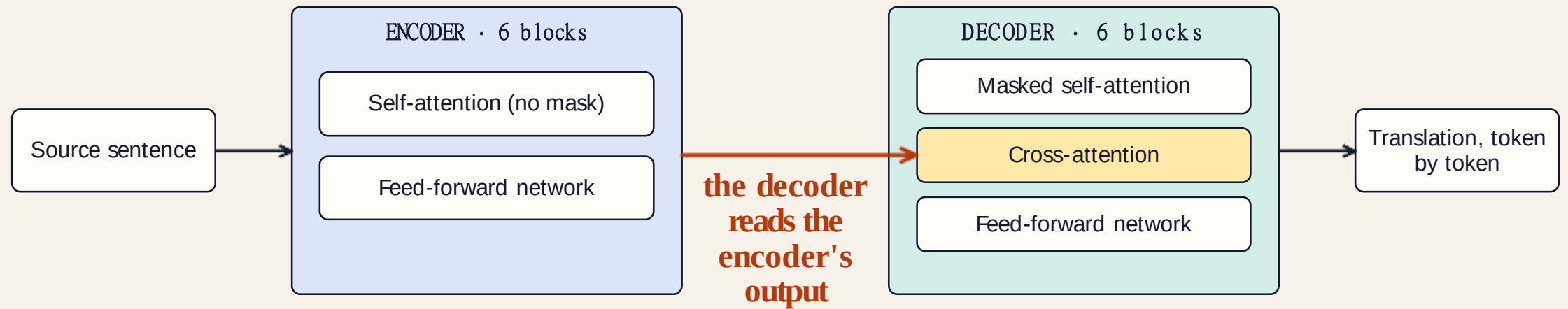
Writers: GPT

Converters: T5

What changed since 2017

Beyond text

# The original encoder–decoder



The 2017 Transformer was a translation machine with two stacks. The encoder reads the whole source sentence; the decoder writes the translation, consulting the encoder through cross-attention.

# Encoder-only models: readers

NO MASK · SEES BOTH DIRECTIONS

The cat [MASK] on the mat

Trained to fill in blanked-out tokens using the words on both sides. The result is a rich understanding of each token in context.

## STRENGTHS

### Good at

Classifying text, search and retrieval, tagging names, producing embeddings of whole passages. Not designed to write.

## MODELS

### Examples

BERT (2018) and its descendants.



# Decoder-only models: writers

CAUSAL MASK · SEES ONLY THE PAST

The cat sat on the ?

Trained to predict the next token. This is the design described throughout this deck, and the one behind today's chat assistants.

## STRENGTHS

### Good at

Generating text of any kind. With the right prompt, also answering, translating, summarising, coding.

## MODELS

### Examples

The GPT series and almost every current large language model.

# Encoder–decoder models: converters

TWO STACKS · READ, THEN WRITE

summarise: The cat sat ...

An unmasked encoder reads the input in full, and a masked decoder writes the output while consulting it through cross-attention.

## STRENGTHS

### Good at

Tasks with a clear input and a distinct output: translation, summarisation, rewriting.

## MODELS

### Examples

The original Transformer (2017), T5 (2019), BART (2019).

# Three families compared

|                               | Encoder-only              | Decoder-only           | Encoder–decoder                       |
|-------------------------------|---------------------------|------------------------|---------------------------------------|
| Can a token see later tokens? | Yes                       | No                     | Yes in the encoder, no in the decoder |
| Training exercise             | Fill in hidden tokens     | Predict the next token | Produce the target text               |
| Natural use                   | Understanding and scoring | Open-ended generation  | Converting one text into another      |
| Cross-attention               | None                      | None                   | Yes                                   |
| Well-known example            | BERT                      | GPT                    | T5                                    |

# Why decoder-only models took over

## ONE SIMPLE EXERCISE

### **Any text is training data**

Next-token prediction needs no paired inputs and outputs. Every document ever written qualifies.

## ONE INTERFACE

### **Every task is “continue this”**

Translation, questions, code: all can be posed as text to be continued, so one model serves them all.

## ONE STACK

### **Simpler to scale**

A single column of identical blocks is easy to enlarge, and the cache makes generation efficient.

Generality won. A model that can continue any text can be steered, through its prompt, into doing almost any language task.

# What has changed since 2017

| Component            | 2017 original                   | Common today                              |
|----------------------|---------------------------------|-------------------------------------------|
| Overall layout       | Encoder and decoder             | Decoder only                              |
| Position information | Fixed waves, added at the input | Rotary encoding, applied inside attention |
| Normalisation        | After each layer                | Before each layer                         |
| Activation switch    | ReLU                            | Smoother variants such as GELU            |
| Context length       | Hundreds of tokens              | Hundreds of thousands, sometimes more     |

Unchanged: attention, feed-forward layers and residual connections. The core has held for years.

# Beyond text

## IMAGES

### Patches as tokens

Cut a picture into a grid of small squares and treat each square as a token. This is the Vision Transformer (2020).

## AUDIO

### Slices of sound

Short stretches of audio are converted to vectors and processed as a sequence, as in modern speech recognition.

## BIOLOGY

### Amino acids as tokens

A protein is a chain of building blocks. Attention-based models such as AlphaFold 2 predict how the chain folds.

## MIXED INPUT

### Several kinds at once

Image patches and word tokens can share one sequence, which is how multimodal assistants read pictures.

Nothing in a transformer is specific to language. Anything that can be cut into a sequence of pieces and turned into vectors can be fed through the same blocks.

# Making transformers cheaper to run

## QUANTISATION

### Fewer digits per dial

Store each parameter with less precision. The model shrinks several-fold with little loss in quality.

## DISTILLATION

### A small student

Train a compact model to imitate a large one's outputs, keeping much of the skill at a fraction of the size.

## FASTER ATTENTION

### Same result, less shuffling

Reorganise the attention arithmetic so the processor moves less data around. FlashAttention (2022) is a well-known example.

## MIXTURE OF EXPERTS

### Only part of the model wakes

Split the feed-forward layer into many specialists and route each token to just a few of them.

None of these changes what a transformer computes in principle. They reduce the memory, time or energy needed to compute it.

# Part 10 in six ideas

01

## Two original halves

An encoder that reads and a decoder that writes, joined by cross-attention.

02

## Encoder-only

No mask; trained to fill blanks; strong at understanding.

03

## Decoder-only

Causal mask; trained to predict the next token; the basis of LLMs.

04

## Why it won

Any text is training data and any task can be phrased as continuation.

05

## Refinements

Rotary positions, pre-norm and smoother activations; the core is unchanged.

06

## General purpose

Images, audio and proteins can all be processed as token sequences.



# LLMs in use

With the mechanism in hand, the everyday behaviour of language models, their strengths and their well-known failings, becomes much easier to explain.

## IN THIS PART

Context window

No memory between calls

Learning from the prompt

Retrieval and tools

Hallucination

Interpretability

# The context window: working memory

## LONG-TERM KNOWLEDGE

### In the weights

Everything absorbed during training. Vast, general, and frozen once training ends.

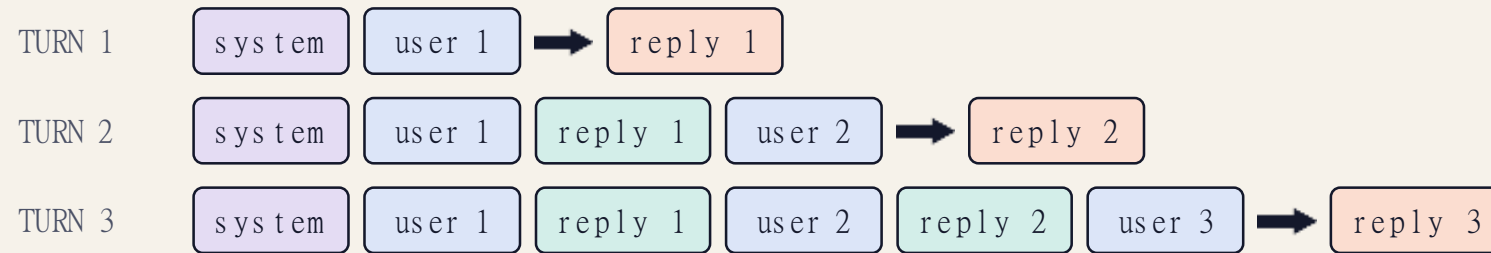
## WORKING MEMORY

### In the context

The tokens in front of the model right now: instructions, conversation, documents. Specific, current, and gone when the conversation ends.

A model can draw on exactly two sources of information. Attention operates only over the second, so anything the model must use precisely should be placed in the context.

# No memory between calls

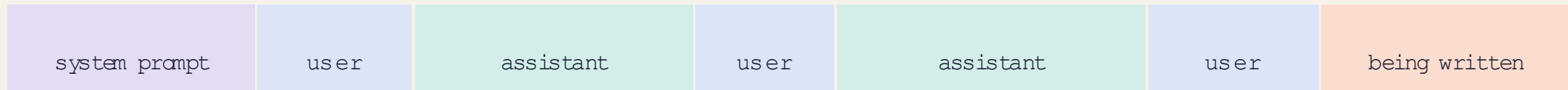


The model keeps nothing from one reply to the next. The chat application resends the entire conversation every time, and the model reads it afresh.

**When a conversation outgrows the context window, the oldest parts must be dropped or summarised.**

# A chat is one long token stream

WHAT THE MODEL RECEIVES ON TURN 3



System instructions, user messages and earlier replies are joined end to end, separated by role tokens. To the model it is a single document to continue.

SETS THE SCENE

## System prompt

Text placed first by the developer: the assistant's role, tone and rules. It has influence because every later token can attend to it.

ONE STREAM

## No separate channels

Instructions and content travel in the same stream, which is why careful prompt design matters.

# In-context learning

A PROMPT WITH EXAMPLES

sea → mer

dog → chien

house → maison

cat → chat

Show a pattern in the prompt and the model continues it, with no retraining.

Attention heads find the earlier examples, work out what links each pair, and apply the same link to the new item. Not one dial changes.

**The pattern lives in the context, not in the weights. Start a new chat and it is gone.**

# Thinking out loud

## DIRECT

### Answer at once

Q: Pens come in packs of 12. How many packs give 40 pupils one pen each?

A: 3

One pass through the blocks must do all the work, and it can come up short.

## STEP BY STEP

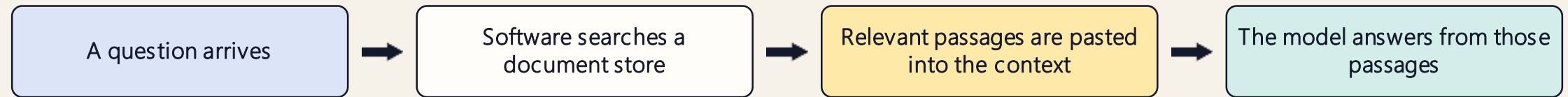
### Work through it first

A: Three packs give 36 pens, which is too few. Four packs give 48, which is enough. So 4 packs.

Each written step becomes context that later tokens can attend to.

A model has a fixed number of blocks per token. Writing out intermediate steps gives it more passes and a scratchpad to lean on.

# Retrieval: an open-book exam



Instead of relying on what the weights happen to remember, give the model the source material and let attention read from it.

## BENEFITS

### Why it helps

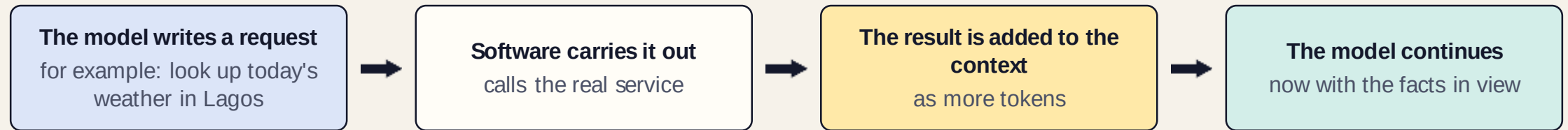
Answers can be current, specific to private documents, and traceable to a source.

## TERM

### The name

Retrieval-augmented generation, usually shortened to RAG.

## Tools: text that triggers actions



The model can only ever produce tokens. Surrounding software watches for specially formatted requests, runs them, and feeds back the outcome.

**Calculators, search, code execution and whole “agents” are built on this loop.**



# Hallucination: fluent is not the same as true

## THE OBJECTIVE

### Trained for plausibility

The loss rewards likely continuations. A confident, well-formed falsehood can be very likely.

## THE MEMORY

### Facts are stored loosely

Knowledge is spread across dials, not filed in records. Rare details are held weakly and can blur together.

## THE LOOP

### Something must be written

Every pass ends with a token being chosen. There is no built-in option to stop and say nothing.

A language model can state things that are false in the same assured tone as things that are true. Retrieval, citations and checking by people reduce the risk; none removes it.

# Other limits to keep in mind

| Limit                                   | Where it comes from                                                                   |
|-----------------------------------------|---------------------------------------------------------------------------------------|
| Knowledge stops at a cut-off date       | The weights were frozen when training ended                                           |
| Only so much text fits                  | The context window, and the steep cost of attention                                   |
| Exact arithmetic and counting are shaky | Tokens are chunks, and the model estimates where a calculator would compute           |
| Answers reflect the training text       | Biases and errors in the data are absorbed along with everything else                 |
| Wording changes the result              | Different prompts produce different vectors, and so different paths through the model |

# Opening the black box

## ATTENTION PATTERNS

### Who looked at whom

Heatmaps like those in Part 6 show how information moved between tokens.

## CIRCUITS

### Small working groups

Sets of heads and units that together carry out an identifiable procedure, such as completing a repeated name.

## FEATURES

### Directions with meaning

Directions in the lanes that respond to a recognisable concept and can sometimes be turned up or down.

We know exactly what arithmetic a transformer performs. Understanding what its billions of learned numbers have come to mean is an active research field, and far from finished.

# Part 11 in six ideas

01

## Two sources

Frozen knowledge in the weights; current material in the context.

02

## Stateless

Each reply is computed from the full conversation, resent every turn.

03

## In-context learning

Patterns shown in the prompt are followed without changing any dial.

04

## Scratchpads

Writing out steps gives the model more passes and more to attend to.

05

## Retrieval and tools

Software places facts and results into the context for the model to use.

06

## Limits

Plausible is not true; knowledge has a cut-off; context is finite.

## PART 12

# Putting it together

Every piece is now on the table. This last part follows one prompt through the whole machine, then gathers the key ideas, a glossary and further reading.

## IN THIS PART

One prompt, end to end

The machine revisited

Ten ideas

Glossary

Further reading

# One prompt, end to end: into numbers

## STAGE 1

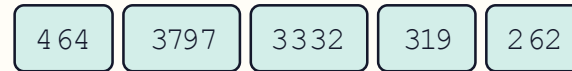
### Tokenise



The tokenizer cuts the prompt into five subword tokens.

## STAGE 2

### Look up IDs



Each token is replaced by its number in the vocabulary.

## STAGE 3

### Embed and stamp



Each ID becomes a meaning vector; a position vector is added. Five lanes are now ready.

Prompt: “The cat sat on the”. The model's task is to produce a probability for every possible next token.

# One prompt, end to end: through the blocks

## STAGE 4

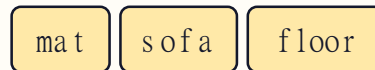
### Attention gathers



The final “the” looks back. Its heads pick out “sat”, “on” and “cat”: something is being sat upon, by a cat.

## STAGE 5

### Feed-forward recalls



Detectors that recognise this situation fire and push the lane towards things a cat sits on.

## STAGE 6

### Repeat, block after block



Each block adds a refinement. By the top, the lane has moved from “the” to “a noun for a surface, coming next”.

Communicate, compute, add back. The same two-step rhythm runs in every block of the stack.

# One prompt, end to end: out to a token

## STAGE 7

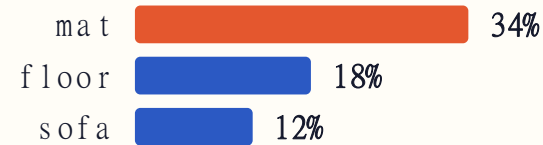
### Unembed



The final vector is compared with every vocabulary entry, giving a logit for each.

## STAGE 8

### Softmax



Logits become probabilities that add up to 100%.

## STAGE 9

### Choose and append

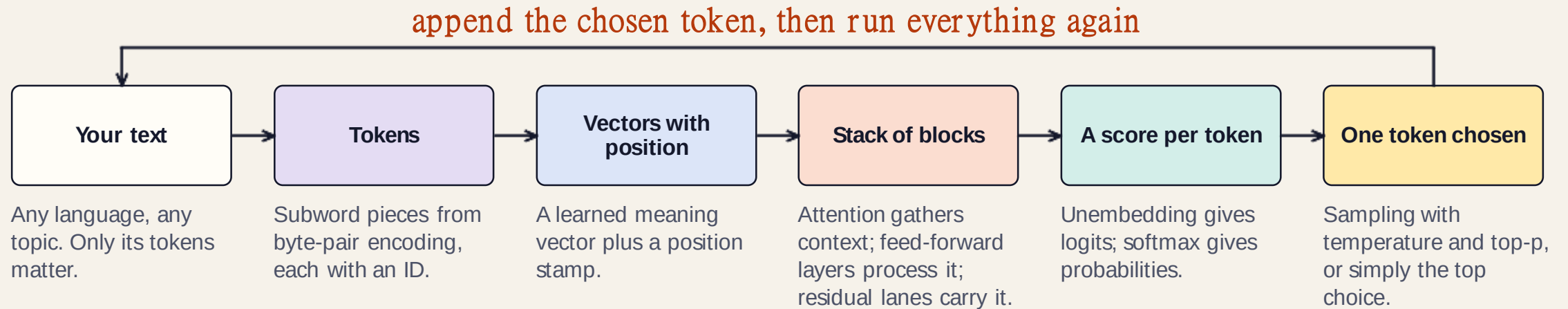


A token is sampled, here “mat”, and joined to the text. Then the loop begins again.

Nine stages, one token. Everything a language model writes is produced by repeating this journey.



# The whole machine, revisited



The picture from the opening, now with every box opened. The same six boxes describe a small research model and the largest assistant alike; only the sizes differ.

# Ten ideas to keep

01

## Next-token prediction

The single task behind everything.

02

## Tokens

Text is cut into subword pieces with IDs.

03

## Embeddings

Each token becomes a learned vector; meaning is geometry.

04

## Position

Order is written into the vectors on purpose.

05

## Attention

Tokens gather context using queries, keys and values.

06

## Causal mask

Each token sees only the past.

07

## Feed-forward layers

Per-token processing and stored knowledge.

08

## Residual lanes

Layers add updates; nothing is overwritten.

09

## Softmax and sampling

Scores become probabilities; one token is drawn.

10

## Training

Guess, measure the penalty, nudge every dial; repeat at vast scale.

# Glossary: A to L

| Term                 | Meaning                                                                |
|----------------------|------------------------------------------------------------------------|
| Attention head       | One set of query, key and value lenses producing one attention pattern |
| Block (layer)        | One round of attention followed by a feed-forward network              |
| Causal mask          | The rule that stops a token attending to later tokens                  |
| Context window       | The maximum number of tokens the model can consider at once            |
| Decoder-only         | A transformer built only from masked blocks; the design of modern LLMs |
| Embedding            | The learned vector that stands for a token                             |
| Feed-forward network | The per-token network in each block: expand, switch, compress          |
| Fine-tuning          | Further training of a pretrained model on a narrower set of examples   |
| Logit                | A raw score for one vocabulary token, before softmax                   |
| Loss                 | The penalty measuring how badly the model predicted the true token     |

# Glossary: P to U

| Term              | Meaning                                                               |
|-------------------|-----------------------------------------------------------------------|
| Parameter         | One adjustable number in the model; a dial                            |
| Position encoding | Information added to vectors so the model knows token order           |
| Query, key, value | What a token seeks, what it advertises, and what it hands over        |
| Residual stream   | The running vector for each token that every layer adds to            |
| Sampling          | Choosing the next token at random according to its probability        |
| Softmax           | The recipe that turns scores into positive shares totalling 100%      |
| Temperature       | A setting that sharpens or flattens the probabilities before sampling |
| Token             | A piece of text, usually a word or part of one, with an ID            |
| Transformer       | An architecture of stacked attention and feed-forward blocks          |
| Unembedding       | Matching the final vector against the vocabulary to produce logits    |

# Where to go next

| Resource                                                                | What it offers                                                                   |
|-------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| “Attention Is All You Need” (Vaswani et al., 2017)                      | The original paper. Short, and readable once the ideas in this deck are familiar |
| “The Illustrated Transformer” (Jay Alammar)                             | A classic picture-led walkthrough of the original encoder–decoder model          |
| 3Blue1Brown’s series on neural networks and transformers                | Animated explanations that build visual intuition for vectors and attention      |
| “Let’s build GPT: from scratch, in code, spelled out” (Andrej Karpathy) | A small working GPT written line by line, for those ready to see the code        |
| “A Mathematical Framework for Transformer Circuits” (Anthropic, 2021)   | An entry point to interpretability and the residual-stream view                  |

A good next step: read the original paper and see how much now looks familiar

## THE WORKING MECHANISM OF A MODERN LLM

Tokens in. Vectors refined, block after block, by attention and feed-forward layers. One token out. Repeat.

That loop, scaled up and trained on a vast amount of text, is what you are talking to when you talk to a language model.